

Table of Contents

1. [Ontology Manager • Overview • Palantir](#)
2. [Ontology Manager • Overview • Palantir](#)
3. [Ontology Manager • Navigation • Palantir](#)
4. [Ontology Manager • Viewing usage • Palantir](#)
5. [Ontology Manager • Migrate to project-based permissions • Palantir](#)
6. [Ontology Manager • Ontology roles migration \[Legacy\] • Palantir](#)
7. [Ontology Manager • Change management • Save changes to the Ontology • Palantir](#)
8. [Ontology Manager • Change management • Review and restore changes • Palantir](#)
9. [Ontology Manager • Export, edit, and import an Ontology • Palantir](#)
10. [Ontology Manager • Ontology cleanup • Palantir](#)

Ontology Manager

Ontology Manager (sometimes called the Ontology Management Application, or OMA) enables you to build and maintain your organization’s Ontology. You can use Ontology Manager for a wide range of activities related to your Ontology, from creating a new object type and defining a new action type, to connecting data to the Ontology, and investigating whether data is updating in user applications.

Accessing the application

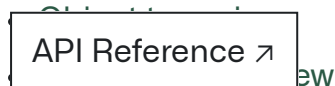
You can access the application in three different ways, by either:

- Selecting the **Ontology Manager** icon from the Workspace sidebar’s **Apps** section;
- Right-clicking on an object type in Data Lineage and selecting **Configure object type**; or
- Adding `/workspace/ontology` to the end of your Foundry home page URL (for instance, `https://example.website.com/workspace/ontology`).

User interface

The Ontology Manager interface is divided into the following elements that you will see referenced throughout the documentation:

- [Ontology Manager navigation](#)
- [Discover view](#)



- [Action type view](#)

Send feedback

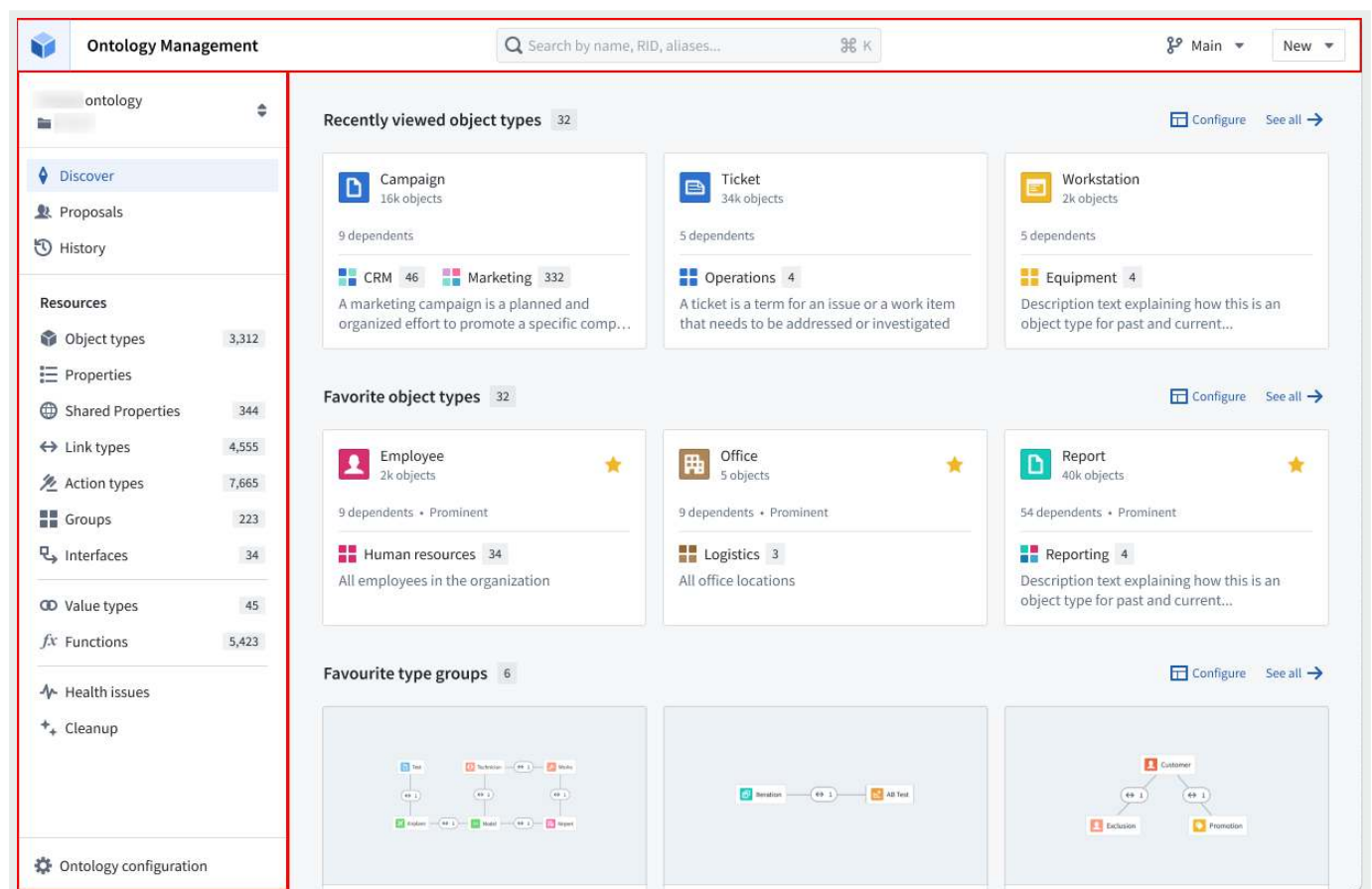
- [Function type view](#)

Ontology Manager navigation

The two persisting elements of Ontology Manager are the top bar and the sidebar. The top bar and sidebar serve as navigation elements, providing intuitive access to various features, functionalities, and sections within the application.

The top bar has three main functionalities. It allows users to search for Ontology resources, create new Ontology resources, and navigate between or create new branches.

The sidebar provides easy navigation to different resources, pages, or applications within Ontology Manager.



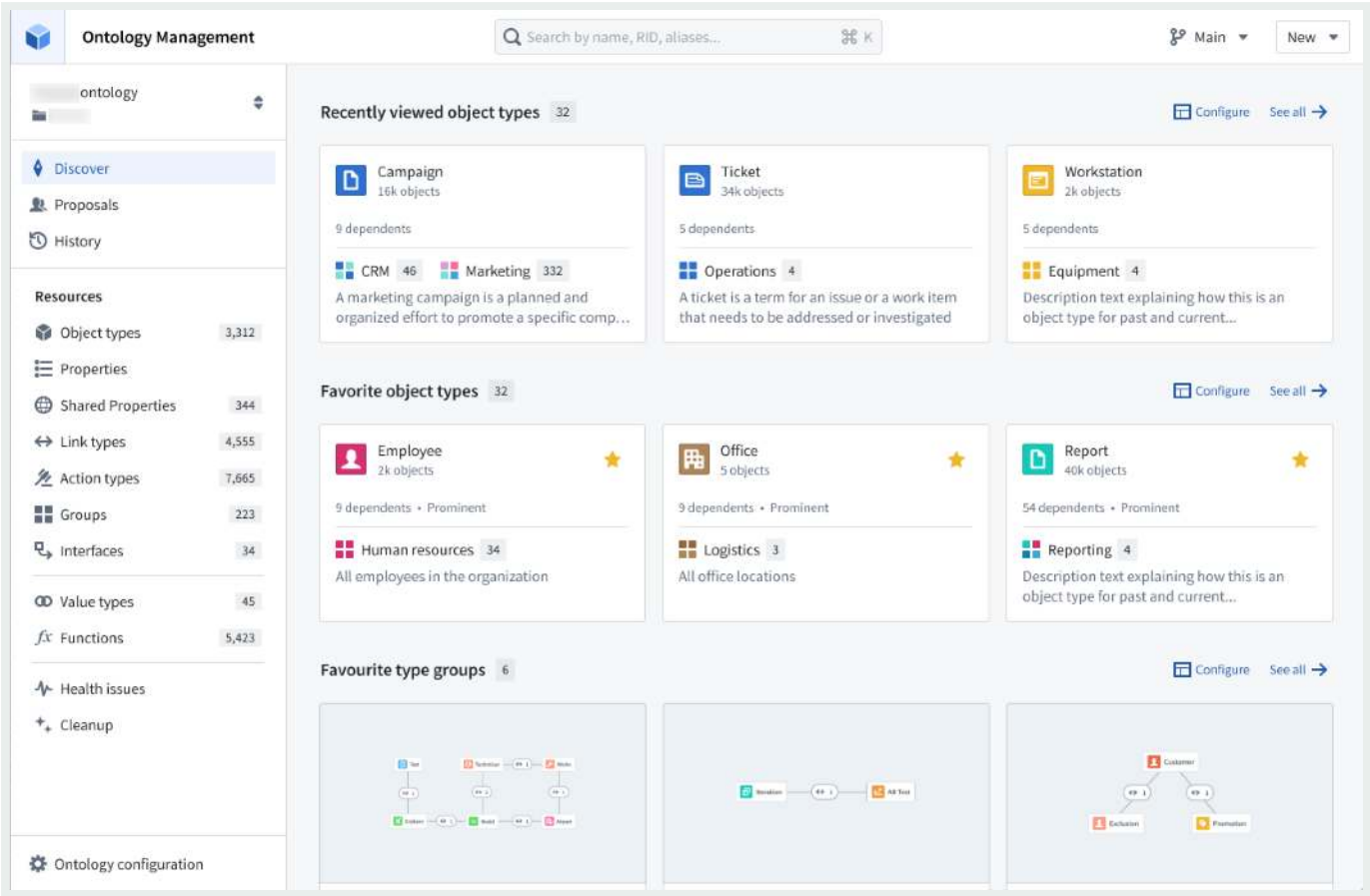
Discover

[API Reference ↗](#)

The Discover view offers a highly customizable landing page tailored to your needs. By default, the Discover view showcases favorite object types, recently-

[Send feedback](#)

and favorite groups.



In case the user is new to the Ontology, two specialized sections will be presented: one which displays all object types that were recently modified within that Ontology, and one for all prominent object types.

API Reference ↗

Send feedback

Ontology Management

Main
New

ontology

Discover

Proposals

History

Resources

Object types 3,312

Properties

Shared Properties 344

Link types 4,555

Action types 7,665

Groups 223

Interfaces 34

Value types 45

Functions 5,423

Health issues

Cleanup

Ontology configuration

Object types recently modified in ontology 32

Inventory 16k objects

9 dependents

Production 332

Represents the stock of car parts. Attributes can be part ID, quantity in stock, location, etc.

Supplier 3k objects

16 dependents

Sales 4

Represents entities that provide raw materials or components. Attributes might i...

Department 2k objects

5 dependents

Equipment 4

Represents different departments within the company. Attributes might include departm...

Transport 16k objects

9 dependents

A mode of transport is a solution that makes use of a certain type of vehicle, infrastructure, and operation.

Order 304k objects

34 dependents

Sales 4 Marketing 332

Represents an order placed by a customer for a specific car part. Attributes could include...

Customer 210k objects

23 dependents

Sales 4

Description text explaining how this is an object type for past and current...

Prominent object types 2

Employee 2k objects

9 dependents • Prominent

Human resources 34

All employees in the organization

Office 5 objects

9 dependents • Prominent

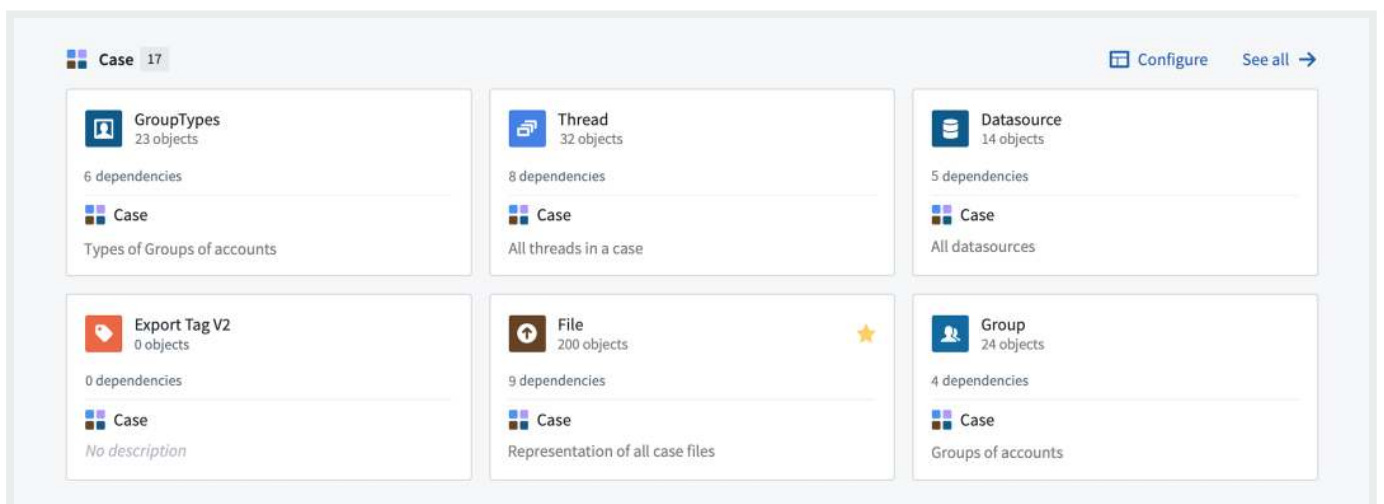
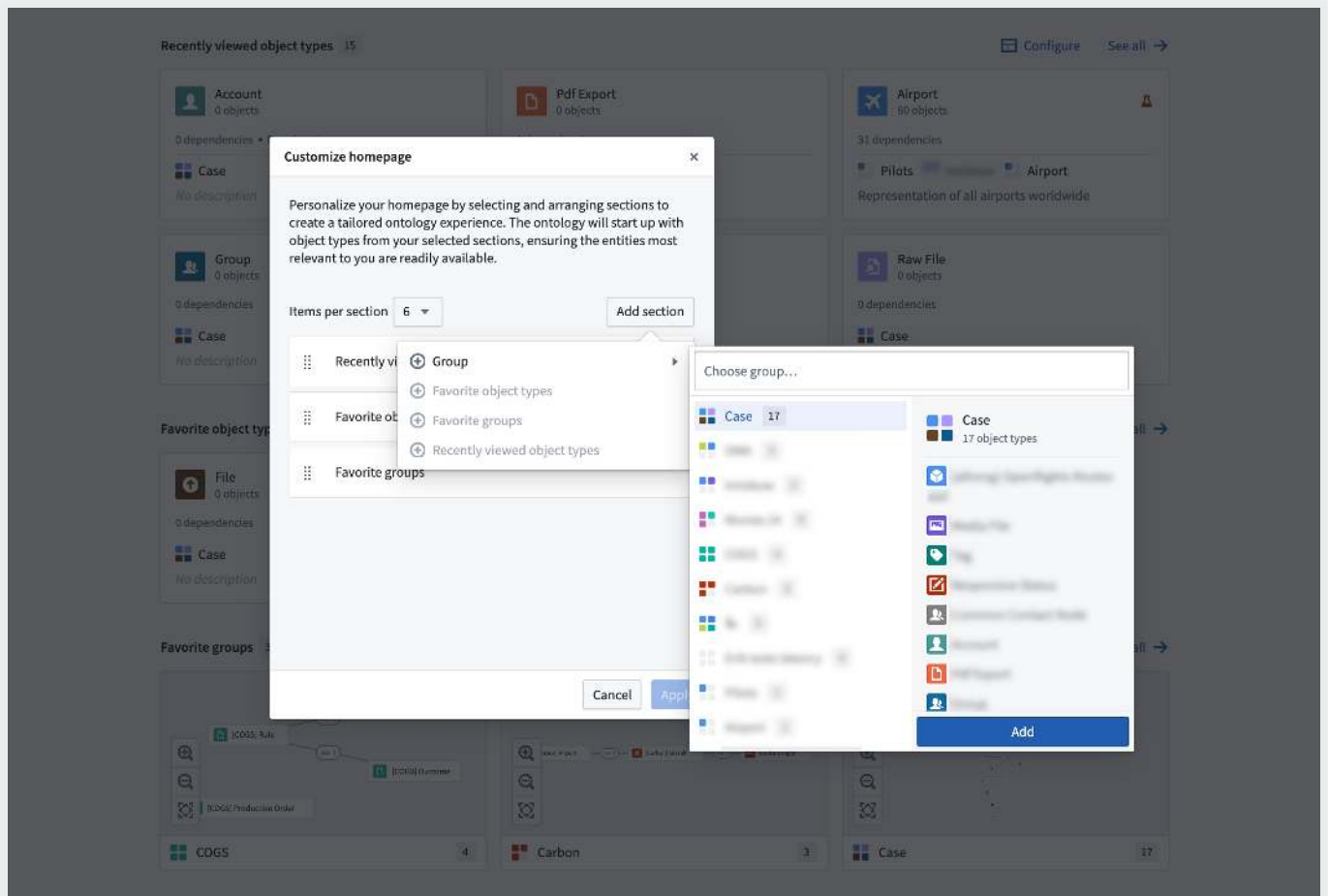
Logistics 3

All office locations

The Discover view provides the flexibility to configure the sections that appear on the page and control the number of items displayed within each section. The available sections include "Recently viewed object types," "Favorite object types," and "Favorite groups." Additionally, you have the option to add a separate section for a specific group, allowing you to explore all object types within that group.

API Reference ↗

Send feedback



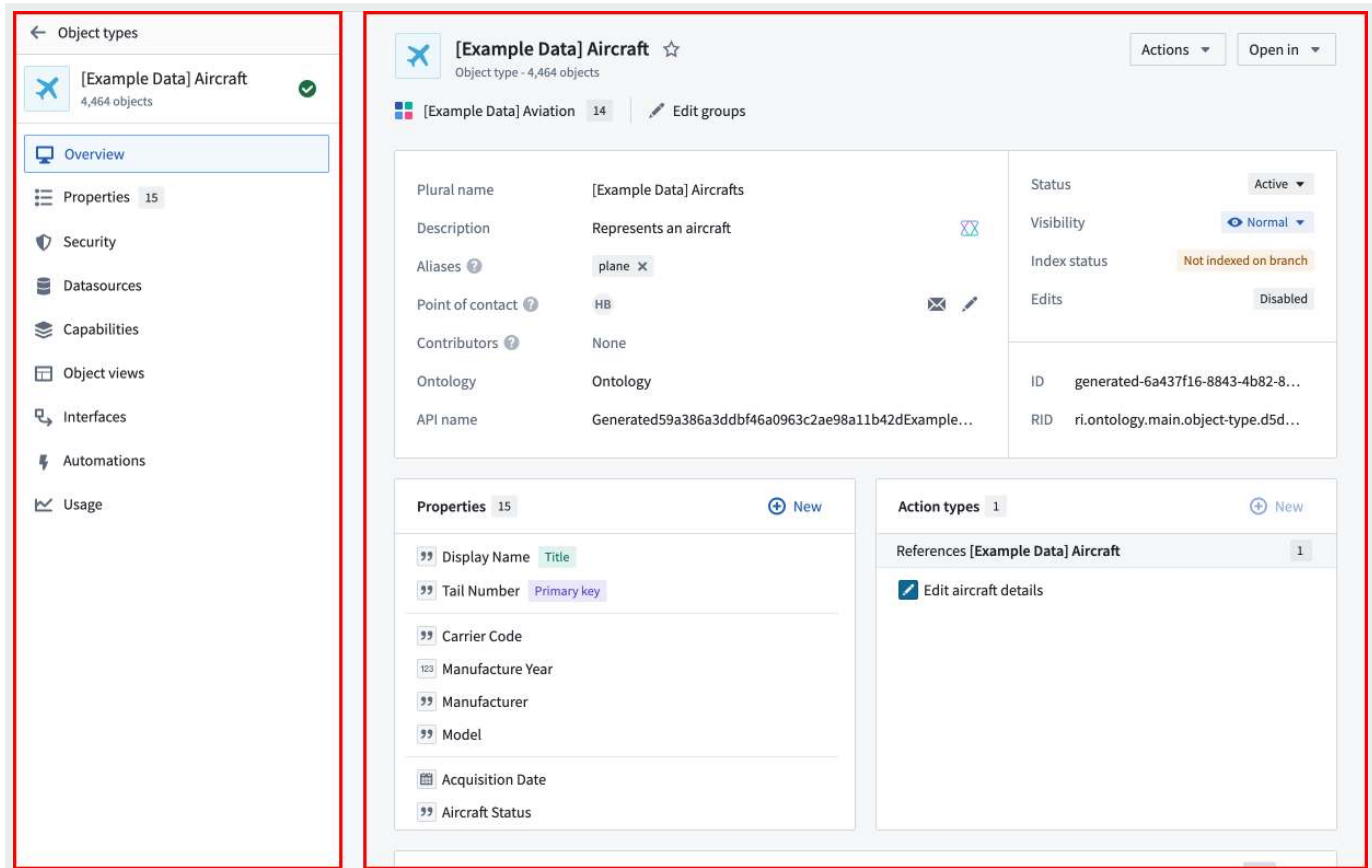
Object type view

Selecting an object type brings up the object type view, which has the following components:

API Reference ↗

- Sidebar with page selections (on the left in the image below)
- Selected page (on the right in the image below)

Send feedback

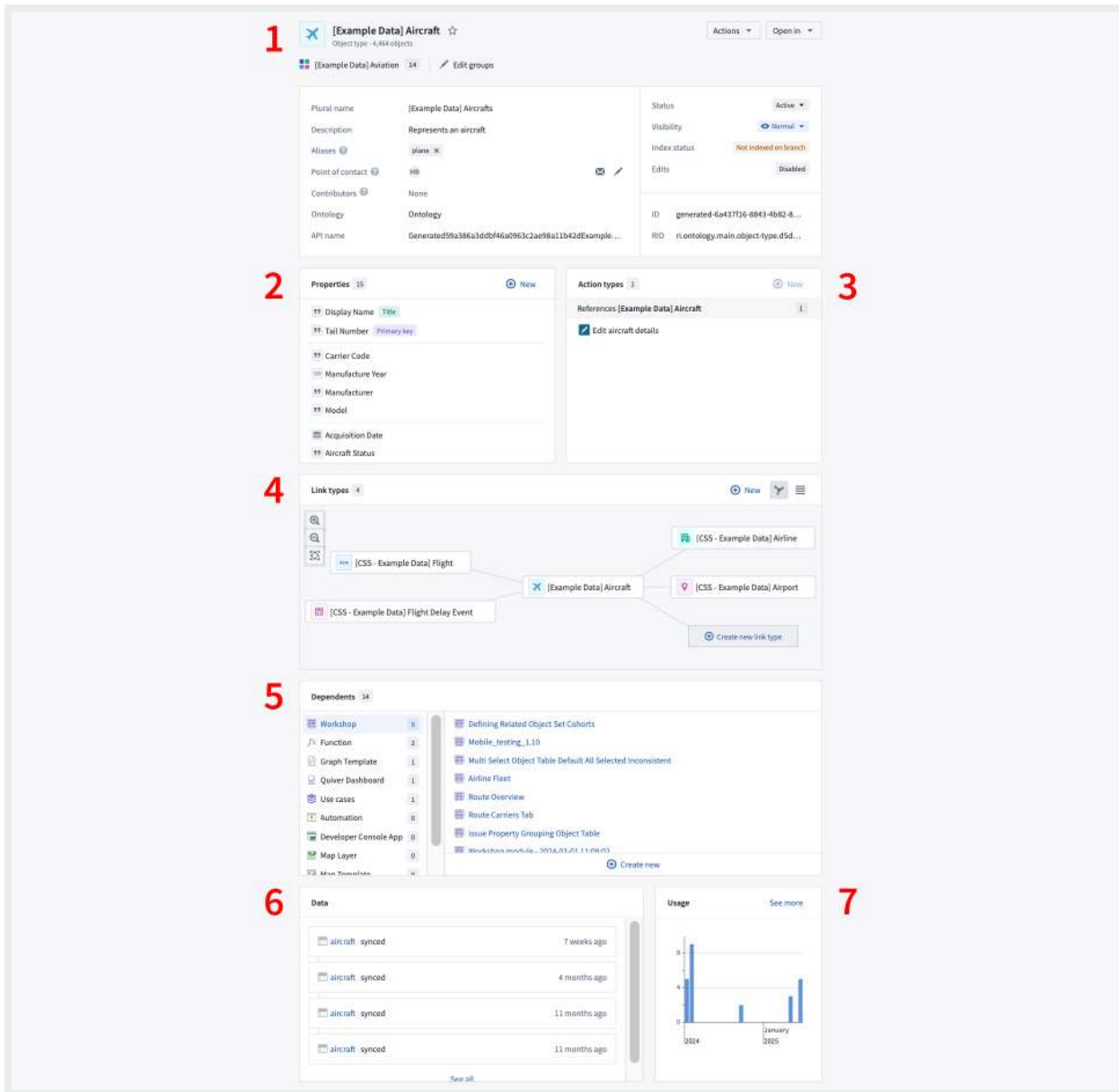


The **Overview** page of an object type has the following sections, as numbered in the image below:

1. Object type metadata
2. Properties
3. Action types
4. Link type graph
5. Dependents
6. Data
7. Usage

API Reference ↗

Send feedback



Property editor view

Select a property from the **Properties** section of an object type's **Overview** page to open the property editor view of the application.

API Reference ↗

Send feedback

Properties

9 of 9 Columns mapped

Properties (13)

Column mapping

Properties 13

Create property

	Property name	Status	Visibility	Base formatter	Column	
☐	Id Primary key	Experimental	Normal	No formatting	id	
☐	ICAO Title	Experimental	Normal	No formatting	ICAO	
☐	New property 4	Experimental	Normal	No formatting		
☐	Airport	Experimental	Normal	No formatting	airport	
☐	City	Experimental	Normal	No formatting	city	
☐	Country	Experimental	Normal	No formatting	country	
☐	Geohash	Experimental	Normal		geohash	
☐	IATA	Experimental	Normal	No formatting	IATA	

airports_ontology_2

Show mapped columns

Automap all

	id	airport	city	country	IATA	ICAO	latitude
	Integer	String	String	String	String	String	Double
	Id Primary key	Airport	City	Country	IATA	ICAO Title	Latitude
1	20	Baltimore/Washington Internatio...	Baltimore	United States	MDW	KBWI	39.17539978
2	65	Shanghai Pudong International Ai...	Shanghai	China	PVG	ZSPD	31.14340019
3	73	Suvarnabhumi Airport	Bangkok	Thailand	BKK	VTBS	13.68109989
4	9	Seattle Tacoma International Airp...	Seattle	United States	SEA	KSEA	47.44900131
5	15	Minneapolis-St Paul International...	Minneapolis	United States	MSP	KMSP	44.88199997
6	22	Salt Lake City International Airport	Salt Lake City	United States	SLC	KSLC	40.78839874

Id Primary key

General

Display

Interaction

Details

Advanced

Overview

Name

Id

Description

Base type

Integer

Allow multiple

Value type

Select a value type

Status

Experimental

Configuration

Title key

Current:

ICAO

Primary key

Preview objects

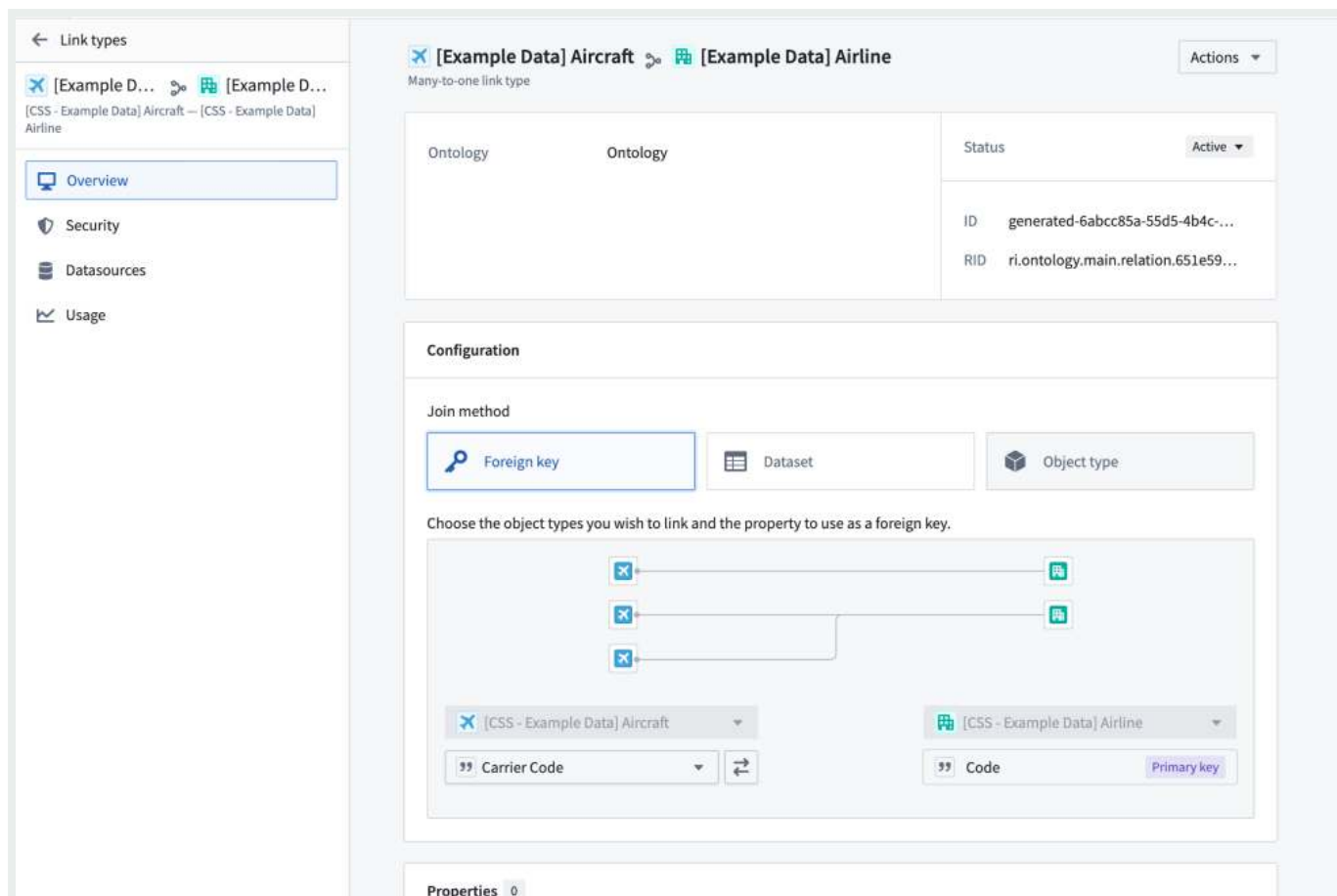
Preview table

Link type view

Selecting a link type from the link type graph of an object type's **Overview** tab (see image below) opens the link type view (with **Overview** and **Datasources** pages).

API Reference ↗

Send feedback

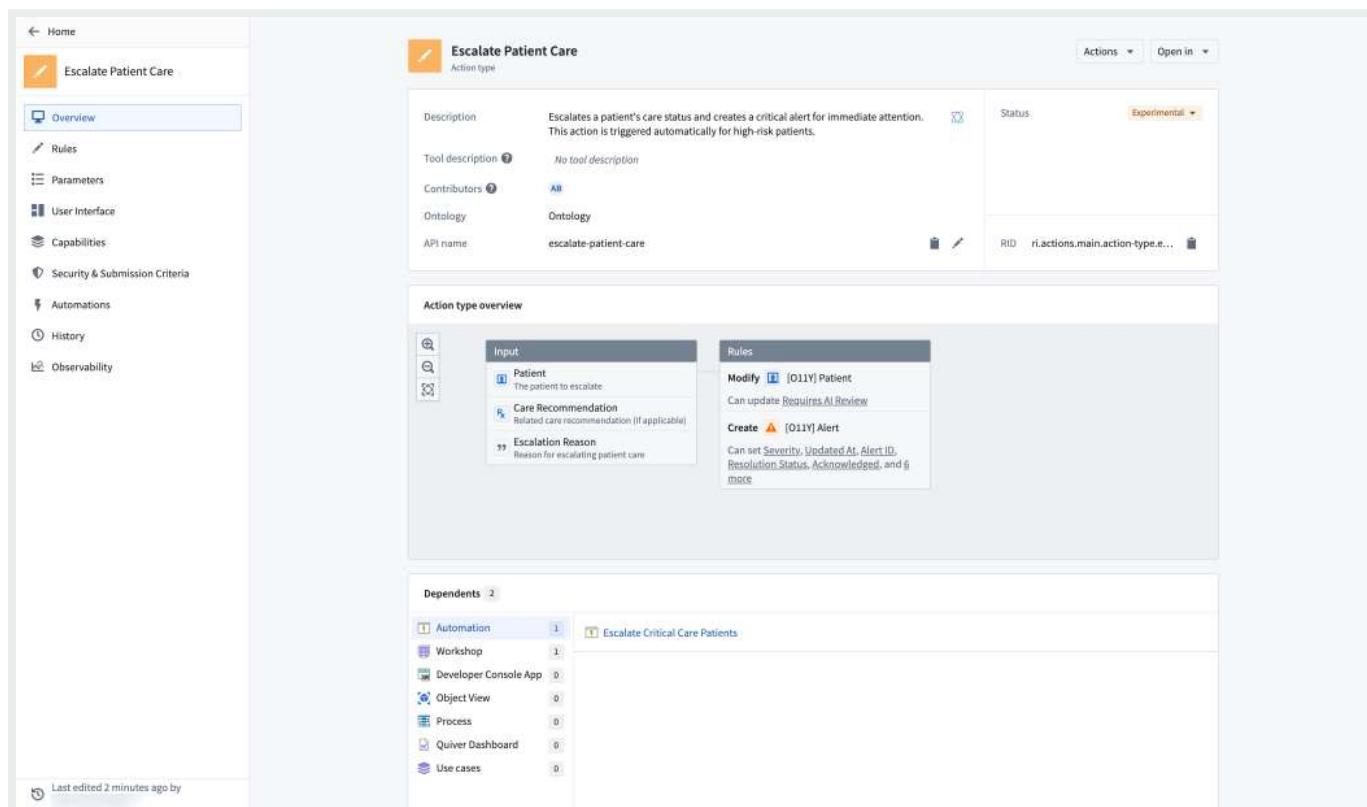


Action type view

Selecting an action type from the action type section of an object type's **Overview** tab opens the action type view, with further access to the **Overview**, **Logic** and **Observability** pages.

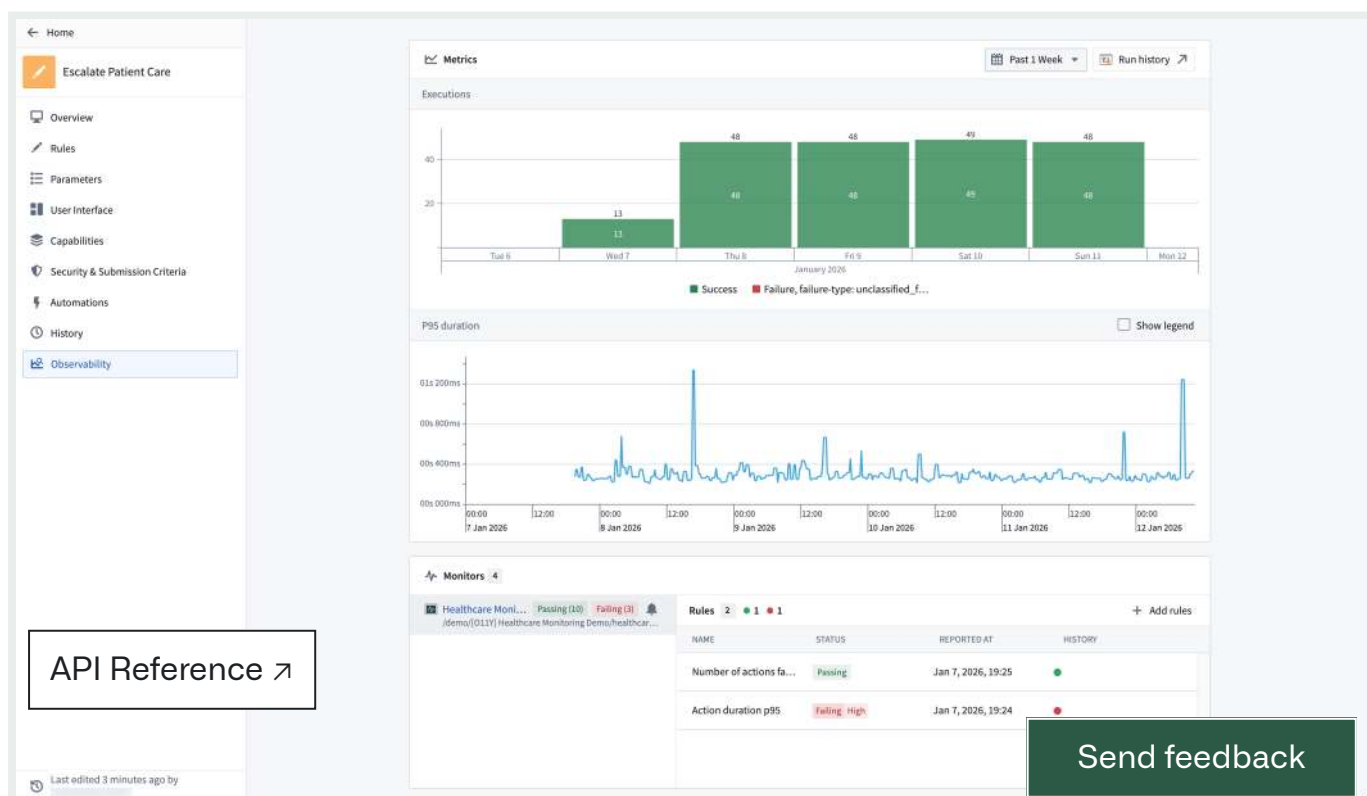
[API Reference ↗](#)

[Send feedback](#)



View action metrics and monitoring rules

The **Observability** tab shows the near real-time usage of the action over the last 30 days as well as any monitoring rules and their status defined for the action. Review action rules for detailed action monitoring rule configuration options.



Function type view

Selecting a function type from the function type section of an object type's **Overview** tab opens the function type view, with further access to the **Overview**, **Configuration** and **Observability** pages.

Usage history of a function

The Usage History panel records the applications which have used any version of a function, along with the respective version information. From this panel, you can navigate to these applications in order to upgrade the version of a function.

View previous versions of a function

By default, the latest version of the function is displayed. To view other versions, use the version dropdown selector located in the left panel.

Navigate to the Functions Code Repository

Modifications to the function can only be made within the Functions Code Repository. To navigate to the repository, use the **Open in Code Repository** button found in the top right-hand corner of the entity view.

[API Reference ↗](#)

[Send feedback](#)

Home

fx

getAgeBasedVitalThresholds

1.1.2 Latest

Overview

Configuration

Observability

Functions in the Ontology Manager are read-only. You can write and modify the function in the Code repositories application.

Open in Code repositories

Name

getAgeBasedVitalThresholds

Visibility

Visible

Type

Default

Published Date

Today at 9:53 AM

Healthcare Monitoring Functions

src/vitalThresholdsCalculators.ts

Class Name

VitalThresholdsCalculator

RID

ri.function-registry.main.function.1686f24c-2542-4081-9a3f-a844f8e86a31

Documentation

Get Age-Based Vital Thresholds Purpose: Returns age-appropriate vital sign thresholds for a patient

Code Preview

Edit in Code Repositories

```

36 @Function()
37 public getAgeBasedVitalThresholds(patient: OilyPatient): VitalThresholds {
38   const patientId = patient.patientId;
39   console.log("Getting vital thresholds for patient", patientId);
40
41   // Call existing calculateAge function (will return negative for future D06)
42   const age = this.helperFunctions.calculateAge(patientId);
43
44   if (age === undefined) {
45     throw new Error("Cannot calculate vital thresholds: Patient S(patientId) has no date of birth");
46   }
47
48   console.log("Patient age:", age);
49
50   // Determine age group and base thresholds
51   let ageGroup: string;
52   let ageGroupIndex: number;
53   let baseThresholds: VitalThresholds;
54
55   if (age < 1) {
56     ageGroup = "Neonate";
57     ageGroupIndex = 0;
58     baseThresholds = {
59       heartRateMin: 100,
60     };
61   }
62 }

```

Inputs

patient

[011] Patient

Output type

View function metrics and monitoring rules

The **Observability** tab shows the near real-time usage of the function over the last 30 days as well as any monitoring rules and their status defined for the function. Review function rules for detailed function monitoring rule configuration options.

Home

fx

getAgeBasedVitalThresholds

1.1.2 Latest

Overview

Configuration

Observability

Functions in the Ontology Manager are read-only. You can write and modify the function in the Code repositories application.

Open in Code repositories

Metrics

Past 1 Week

Run history

Executions

Day	Success	Failure, failure-type: runtime_error
Tue 6	24	0
Wed 7	31	0
Thu 8	48	0
Fri 9	47	0
Sat 10	48	0
Sun 11	47	0
Mon 12	47	0

P95 duration

Show legend

Monitors

Monitor Patient F... Passing (4) Failing (0)

Rules 2 2 0

Add rules

NAME	STATUS	REPORTED AT	HISTORY
Number of function f...	Passing	Jan 7, 2026, 20:38	
Function duration p95	Passing	Jan 7, 2026, 20:39	

Send feedback

API Reference ↗

PREVIOUS

← [Object Views / Add Object Views to a Marketplace product](#)

NEXT

[Navigation](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology Manager

Ontology Manager (sometimes called the Ontology Management Application, or OMA) enables you to build and maintain your organization’s Ontology. You can use Ontology Manager for a wide range of activities related to your Ontology, from creating a new object type and defining a new action type, to connecting data to the Ontology, and investigating whether data is updating in user applications.

Accessing the application

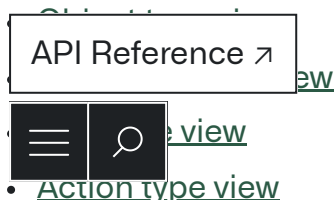
You can access the application in three different ways, by either:

- Selecting the **Ontology Manager** icon from the Workspace sidebar’s **Apps** section;
- Right-clicking on an object type in Data Lineage and selecting **Configure object type**; or
- Adding `/workspace/ontology` to the end of your Foundry home page URL (for instance, `https://example.website.com/workspace/ontology`).

User interface

The Ontology Manager interface is divided into the following elements that you will see referenced throughout the documentation:

- [Ontology Manager navigation](#)
- [Discover view](#)



Send feedback

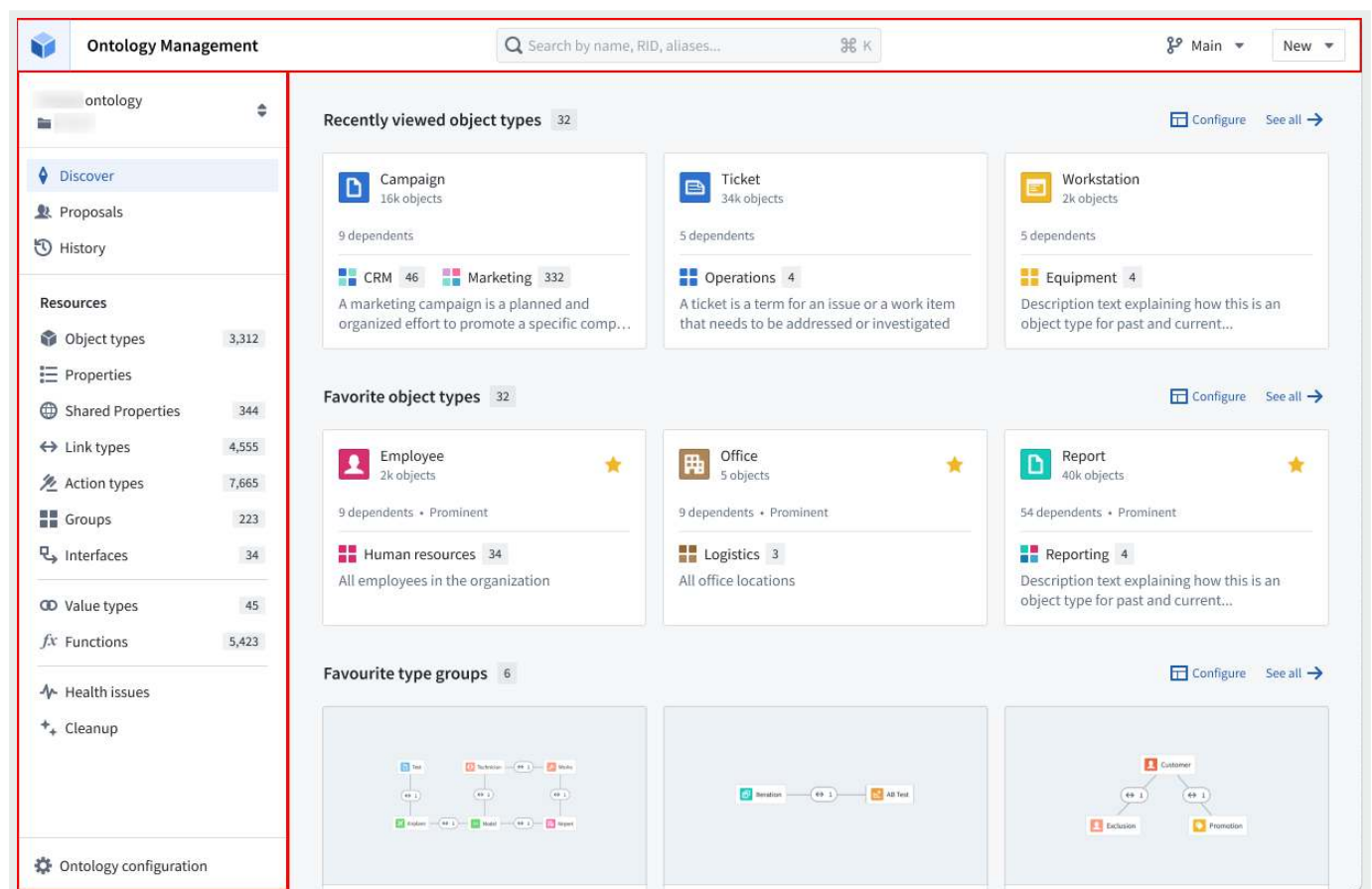
- [Function type view](#)

Ontology Manager navigation

The two persisting elements of Ontology Manager are the top bar and the sidebar. The top bar and sidebar serve as navigation elements, providing intuitive access to various features, functionalities, and sections within the application.

The top bar has three main functionalities. It allows users to search for Ontology resources, create new Ontology resources, and navigate between or create new branches.

The sidebar provides easy navigation to different resources, pages, or applications within Ontology Manager.



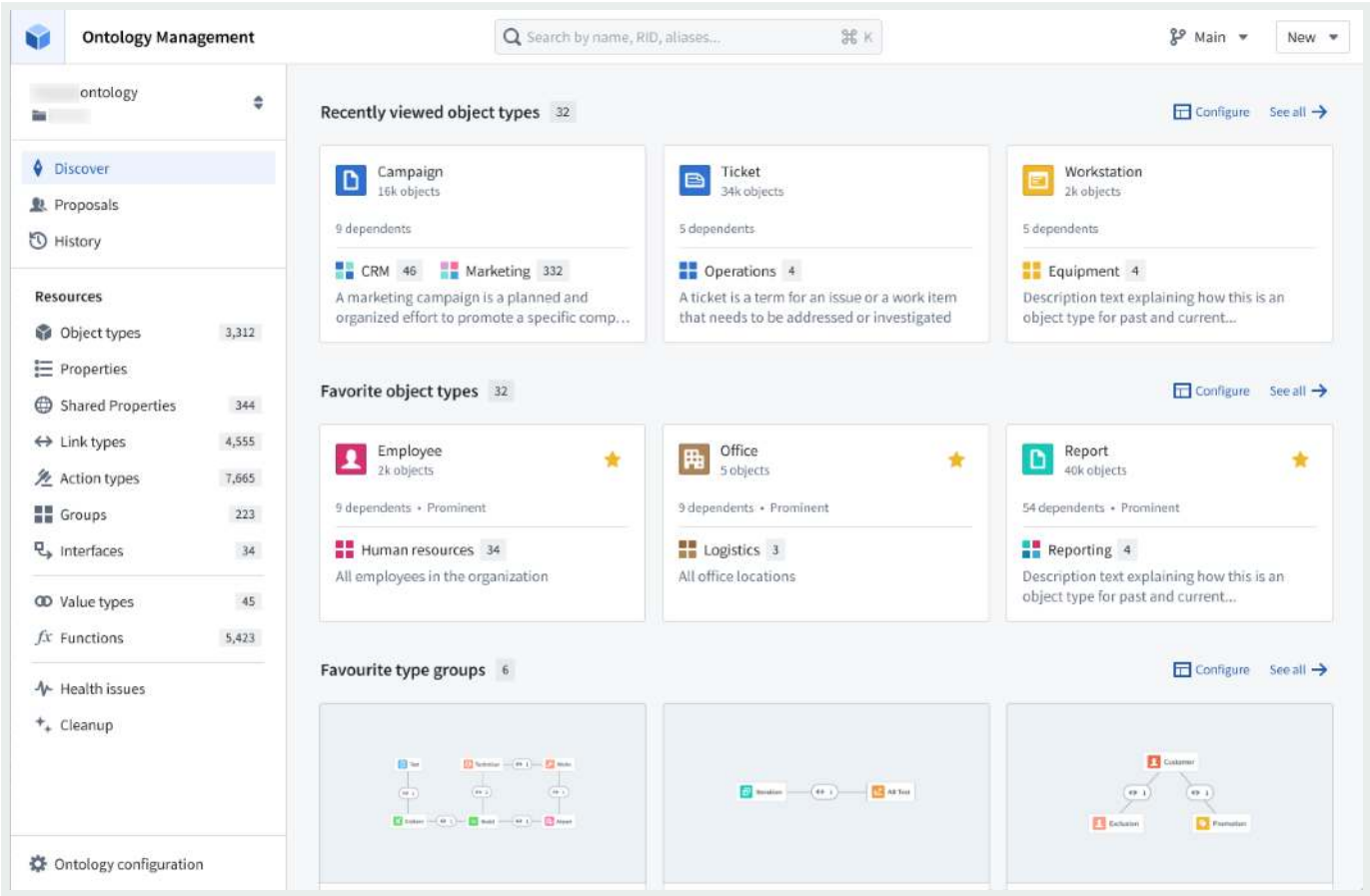
Discover

[API Reference ↗](#)

The Discover view offers a highly customizable landing page tailored to your needs. By default, the Discover view showcases favorite object types, recently-

[Send feedback](#)

and favorite groups.



In case the user is new to the Ontology, two specialized sections will be presented: one which displays all object types that were recently modified within that Ontology, and one for all prominent object types.

API Reference ↗

Send feedback

Ontology Management

Main
New

ontology

Discover

Proposals

History

Resources

Object types

3,312

Properties

Shared Properties

344

Link types

4,555

Action types

7,665

Groups

223

Interfaces

34

Value types

45

Functions

5,423

Health issues

Cleanup

Ontology configuration

Object types recently modified in

ontology

32

Configure

See all

Inventory

16k objects

9 dependents

Production

332

Represents the stock of car parts. Attributes can be part ID, quantity in stock, location, etc.

Supplier

3k objects

16 dependents

Sales

4

Represents entities that provide raw materials or components. Attributes might i...

Department

2k objects

5 dependents

Equipment

4

Represents different departments within the company. Attributes might include departm...

Transport

16k objects

9 dependents

A mode of transport is a solution that makes use of a certain type of vehicle, infrastructure, and operation.

Order

304k objects

34 dependents

Sales

4

Marketing

332

Represents an order placed by a customer for a specific car part. Attributes could include...

Customer

210k objects

23 dependents

Sales

4

Description text explaining how this is an object type for past and current...

Prominent object types

2

Configure

See all

Employee

2k objects

9 dependents • Prominent

Human resources

34

All employees in the organization

Office

5 objects

9 dependents • Prominent

Logistics

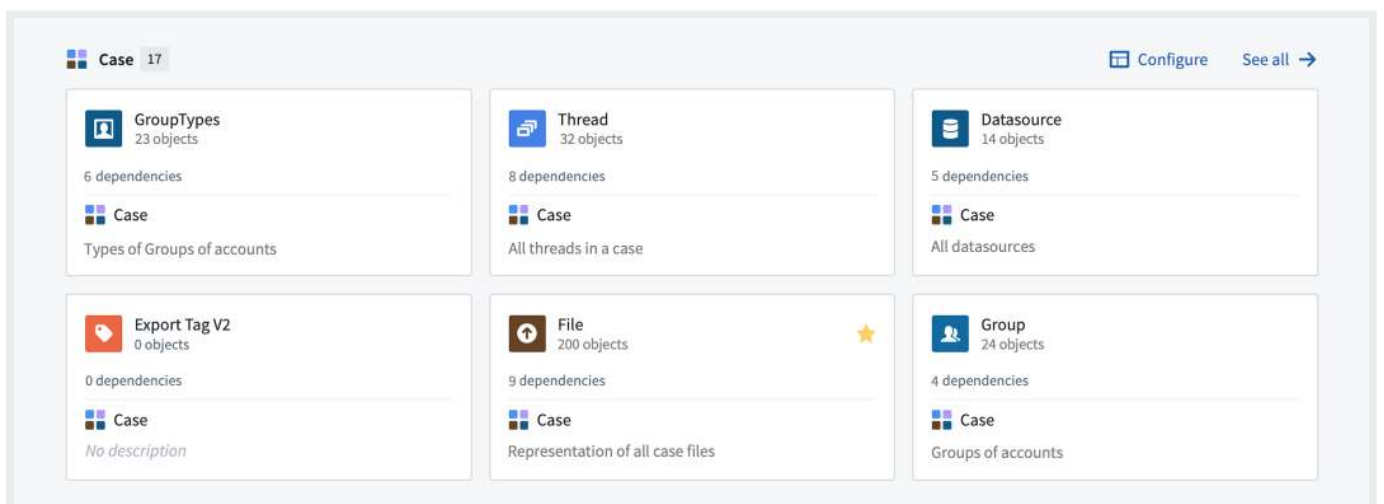
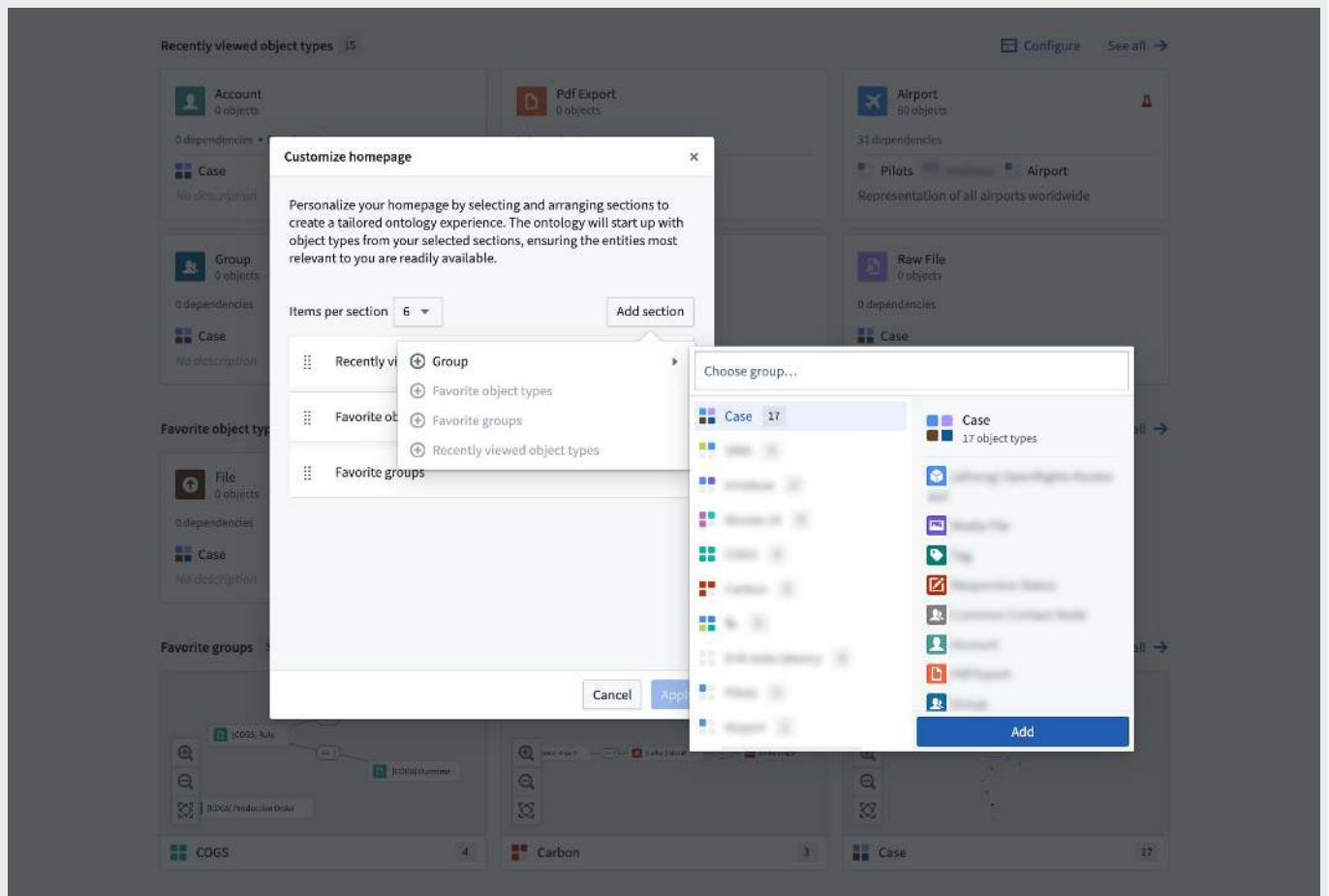
3

All office locations

The Discover view provides the flexibility to configure the sections that appear on the page and control the number of items displayed within each section. The available sections include "Recently viewed object types," "Favorite object types," and "Favorite groups." Additionally, you have the option to add a separate section for a specific group, allowing you to explore all object types within that group.

API Reference ↗

Send feedback



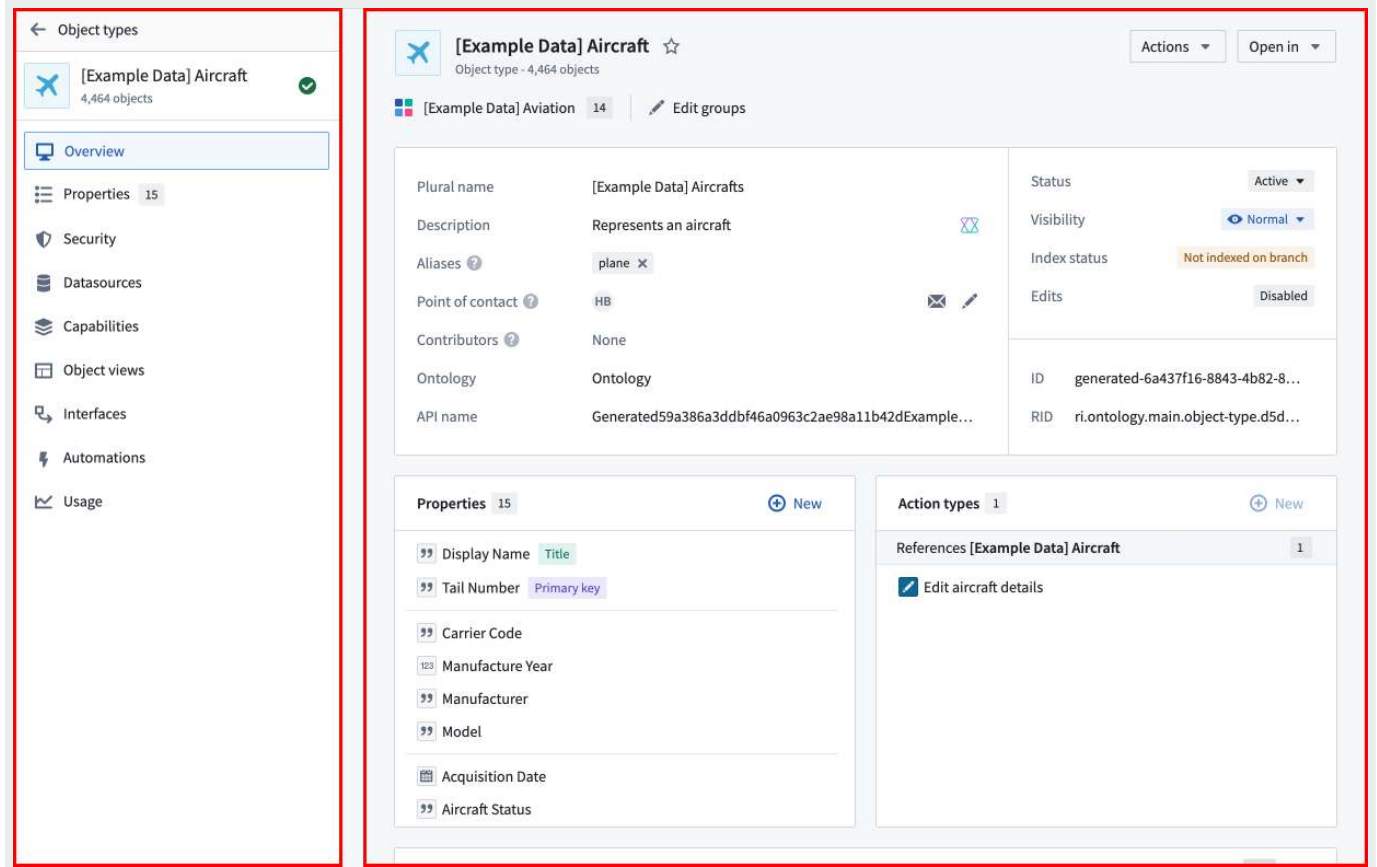
Object type view

Selecting an object type brings up the object type view, which has the following components:

API Reference ↗

- Sidebar with page selections (on the left in the image below)
- Selected page (on the right in the image below)

Send feedback

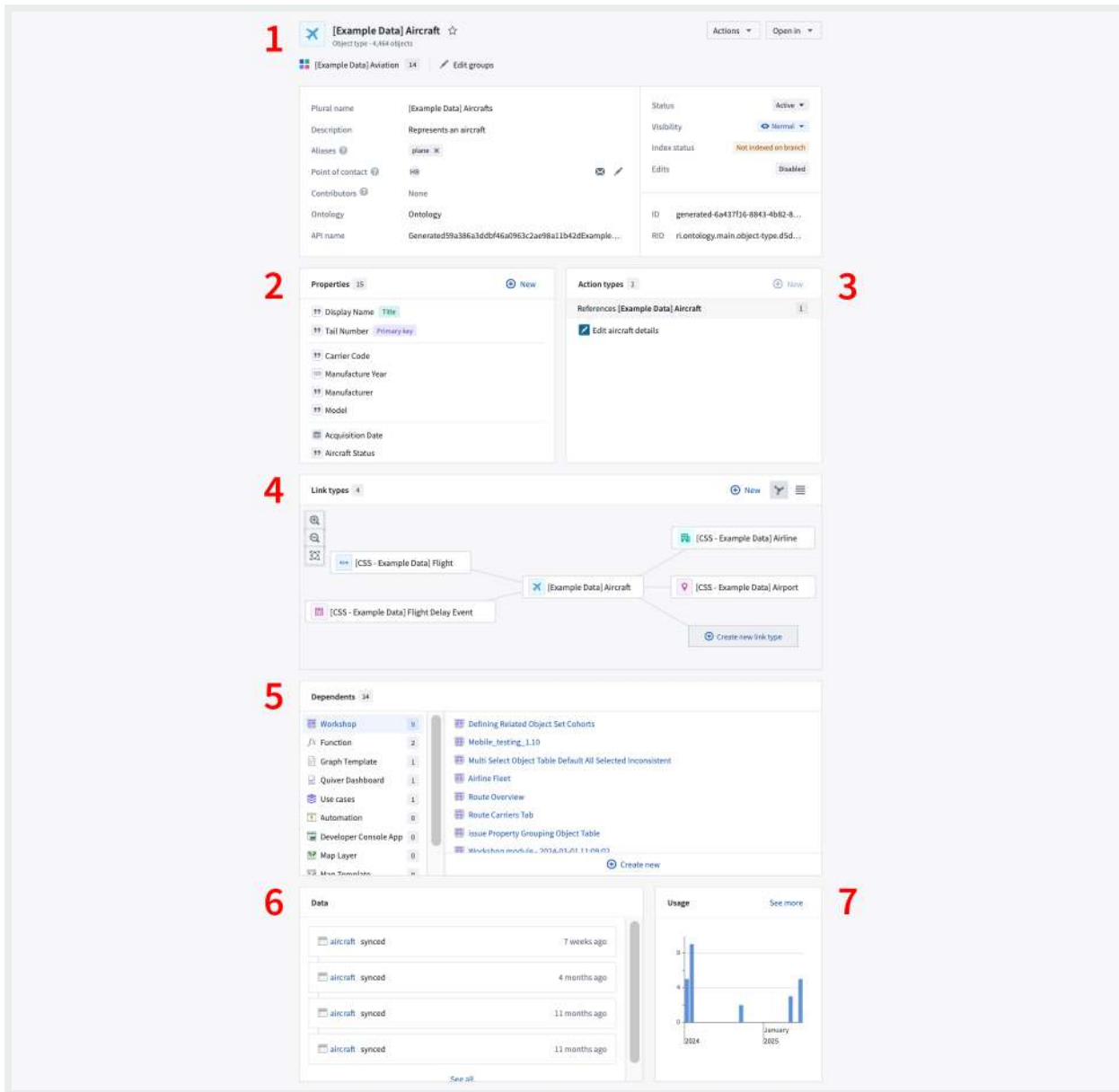


The **Overview** page of an object type has the following sections, as numbered in the image below:

1. Object type metadata
2. Properties
3. Action types
4. Link type graph
5. Dependents
6. Data
7. Usage

[API Reference ↗](#)

[Send feedback](#)



Property editor view

Select a property from the **Properties** section of an object type's **Overview** page to open the property editor view of the application.

API Reference ↗

Send feedback

Properties

9 of 9 Columns mapped

Properties (13)

Column mapping

Properties 13

Create property

Property name	Status	Visibility	Base formatter	Column
☐ Id Primary key	Experimental	Normal	No formatting	id
☐ ICAO Title	Experimental	Normal	No formatting	ICAO
☐ New property 4	Experimental	Normal	No formatting	
☐ Airport	Experimental	Normal	No formatting	airport
☐ City	Experimental	Normal	No formatting	city
☐ Country	Experimental	Normal	No formatting	country
☐ Geohash	Experimental	Normal		geohash
☐ IATA	Experimental	Normal	No formatting	IATA

airports_ontology_2

Show mapped columns

Automap all

	id	airport	city	country	IATA	ICAO	latitude
	Integer	String	String	String	String	String	Double
	Id Primary key	Airport	City	Country	IATA	ICAO Title	Latitude
1	20	Baltimore/Washington Internatio...	Baltimore	United States	MDW	KBWI	39.17539978
2	65	Shanghai Pudong International Ai...	Shanghai	China	PVG	ZSPD	31.14340019
3	73	Suvarnabhumi Airport	Bangkok	Thailand	BKK	VTBS	13.68109989
4	9	Seattle Tacoma International Airp...	Seattle	United States	SEA	KSEA	47.44900131
5	15	Minneapolis-St Paul International...	Minneapolis	United States	MSP	KMSP	44.88199997
6	22	Salt Lake City International Airport	Salt Lake City	United States	SLC	KSLC	40.78839874

Id Primary key

General

Display

Interaction

Details

Advanced

Overview

Name

Id

Description

Base type

Integer

Allow multiple

Value type

Select a value type

Status

Experimental

Configuration

Title key

Current: ICAO

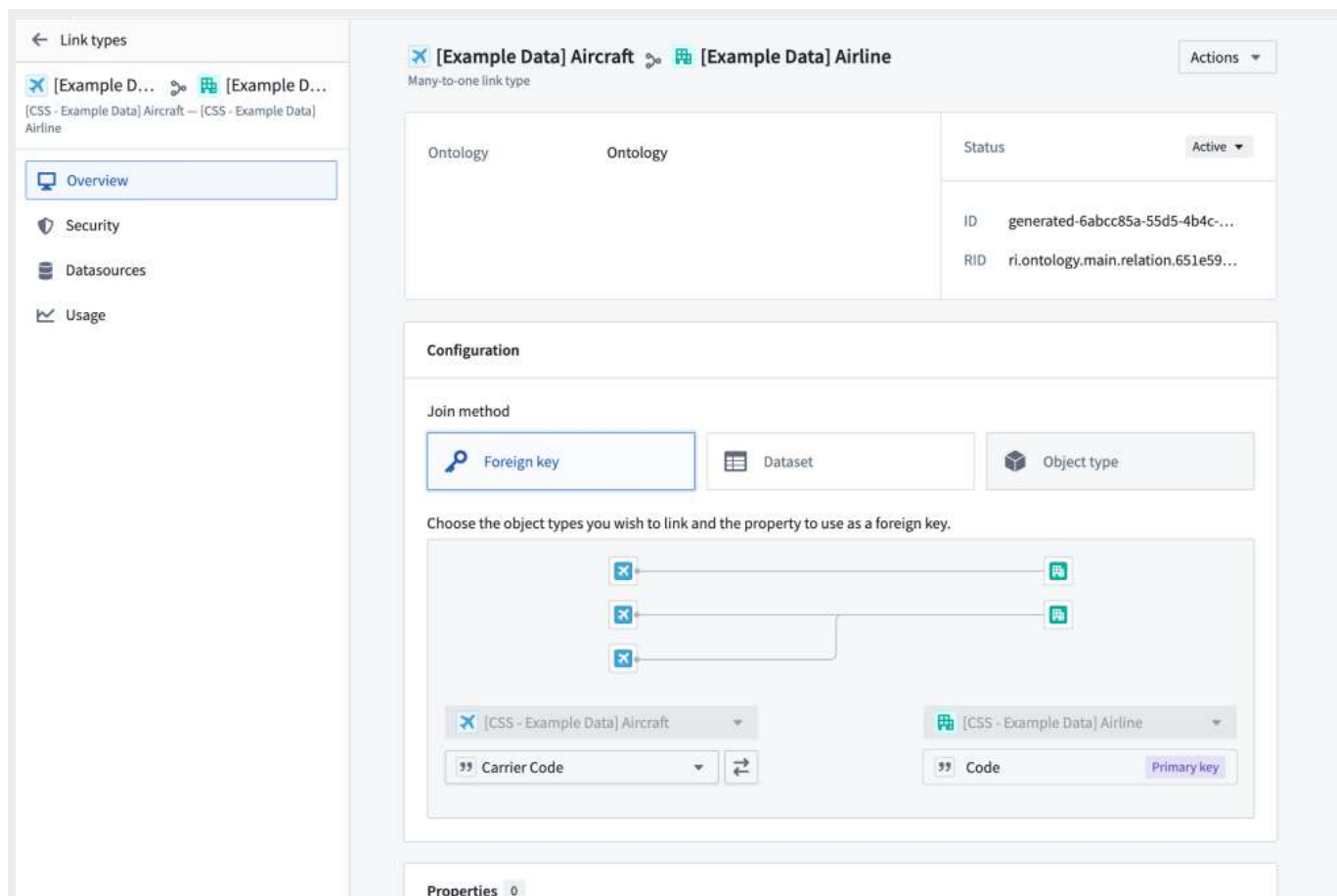
Primary key

Link type view

Selecting a link type from the link type graph of an object type's **Overview** tab (see image below) opens the link type view (with **Overview** and **Datasources** pages).

API Reference ↗

Send feedback

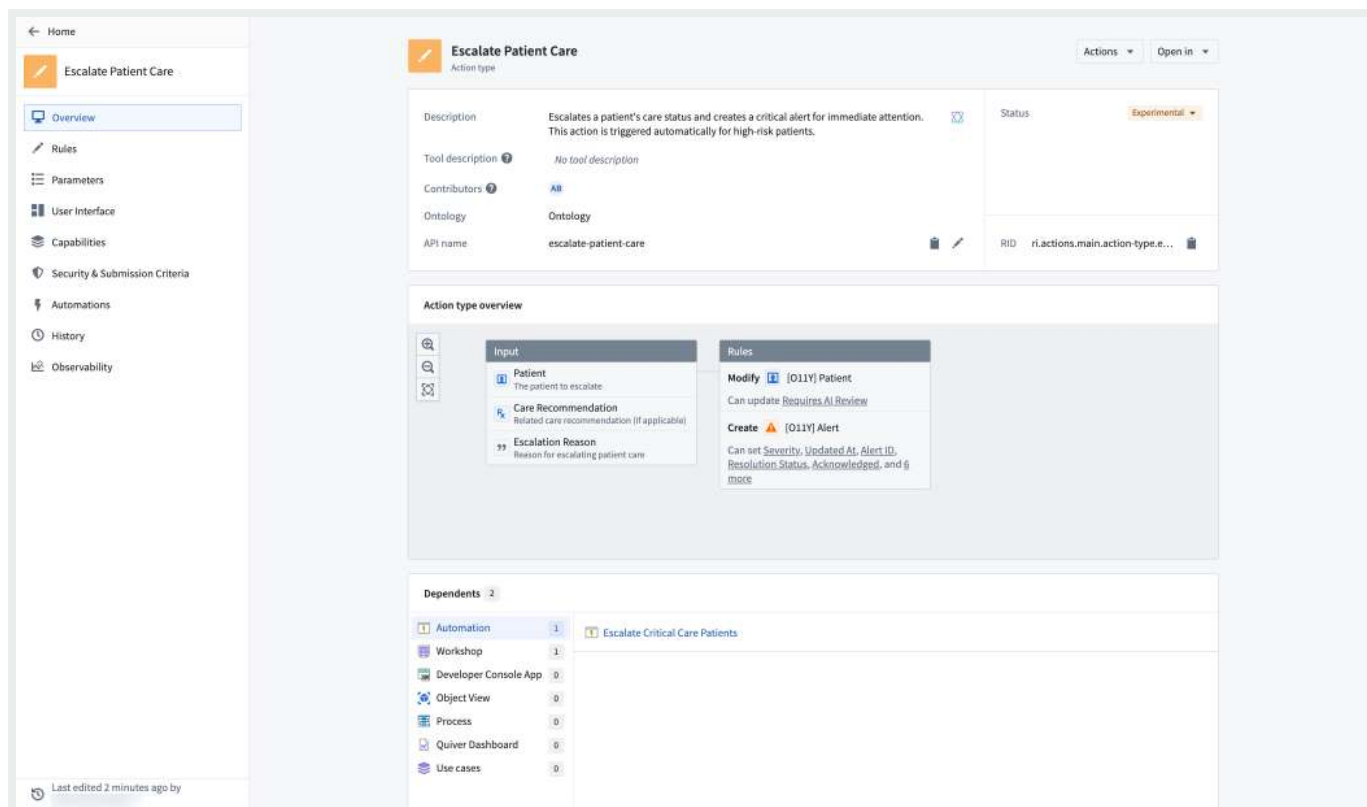


Action type view

Selecting an action type from the action type section of an object type's **Overview** tab opens the action type view, with further access to the **Overview**, **Logic** and **Observability** pages.

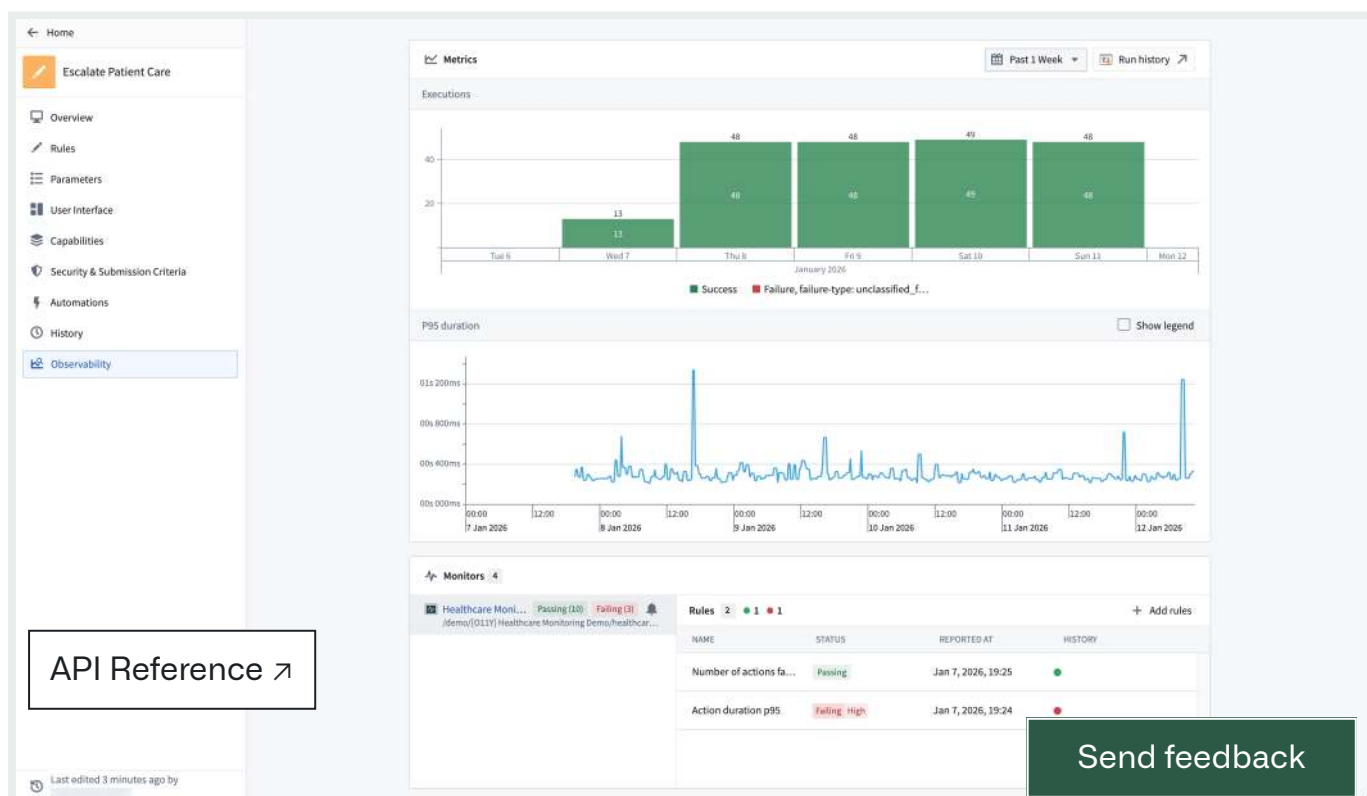
[API Reference ↗](#)

[Send feedback](#)



View action metrics and monitoring rules

The **Observability** tab shows the near real-time usage of the action over the last 30 days as well as any monitoring rules and their status defined for the action. Review action rules for detailed action monitoring rule configuration options.



Function type view

Selecting a function type from the function type section of an object type's **Overview** tab opens the function type view, with further access to the **Overview**, **Configuration** and **Observability** pages.

Usage history of a function

The Usage History panel records the applications which have used any version of a function, along with the respective version information. From this panel, you can navigate to these applications in order to upgrade the version of a function.

View previous versions of a function

By default, the latest version of the function is displayed. To view other versions, use the version dropdown selector located in the left panel.

Navigate to the Functions Code Repository

Modifications to the function can only be made within the Functions Code Repository. To navigate to the repository, use the **Open in Code Repository** button found in the top right-hand corner of the entity view.

[API Reference ↗](#)

[Send feedback](#)

Home

fx

getAgeBasedVitalThresholds

1.1.2 Latest

Overview

Configuration

Observability

Functions in the Ontology Manager are read-only. You can write and modify the function in the Code repositories application.

Open in Code repositories

Name

getAgeBasedVitalThresholds

Visibility

Visible

Type

Default

Published Date

Today at 9:53 AM

Documentation

Get Age-Based Vital Thresholds Purpose: Returns age-appropriate vital sign thresholds for a patient

Healthcare Monitoring Functions

src/vitalThresholdsCalculators.ts

Class Name

VitalThresholdsCalculator

RID

ri.function-registry.main.function.1686f24c-2542-4081-9a3f-a844f8e86a31

Code Preview

Edit in Code Repositories

```

36 @Function()
37 public getAgeBasedVitalThresholds(patient: OilyPatient): VitalThresholds {
38   const patientId = patient.patientId;
39   console.log("Getting vital thresholds for patient", patientId);
40
41   // Call existing calculateAge function (will return negative for future D06)
42   const age = this.helperFunctions.calculateAge(patientId);
43
44   if (age === undefined) {
45     throw new Error("Cannot calculate vital thresholds: Patient S(patientId) has no date of birth");
46   }
47
48   console.log("Patient age:", age);
49
50   // Determine age group and base thresholds
51   let ageGroup: string;
52   let ageGroupIndex: number;
53   let baseThresholds: VitalThresholds;
54
55   if (age < 1) {
56     ageGroup = "Neonate";
57     ageGroupIndex = 0;
58     baseThresholds = {
59       heartRateMin: 100,

```

Inputs

patient

[011] Patient

Output type

View function metrics and monitoring rules

The **Observability** tab shows the near real-time usage of the function over the last 30 days as well as any monitoring rules and their status defined for the function. Review function rules for detailed function monitoring rule configuration options.

Home

fx

getAgeBasedVitalThresholds

1.1.2 Latest

Overview

Configuration

Observability

Functions in the Ontology Manager are read-only. You can write and modify the function in the Code repositories application.

Open in Code repositories

Metrics

Past 1 Week

Run history

Executions

P95 duration

Show legend

Monitors

Monitor Patient F... Passing (4) Failing (0)

Rules 2 2 0

Add rules

NAME	STATUS	REPORTED AT	HISTORY
Number of function f...	Passing	Jan 7, 2026, 20:38	
Function duration p95	Passing	Jan 7, 2026, 20:39	

API Reference ↗

Send feedback

PREVIOUS

← [Object Views / Add Object Views to a Marketplace product](#)

NEXT

[Navigation](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

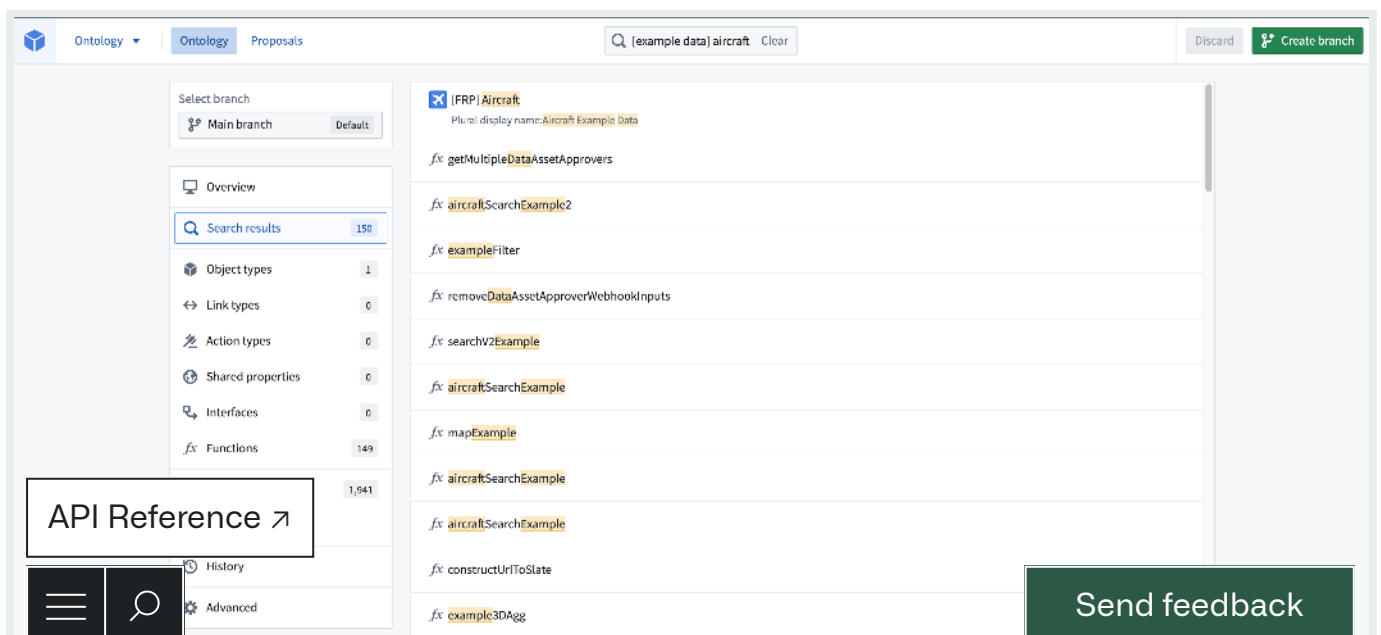
Navigation

Header search bar

In the middle of the header of the application is the **Search** bar. You can click into the **Search** bar or hold down `Cmd/Ctrl + K` to open the **Search** bar dialog.

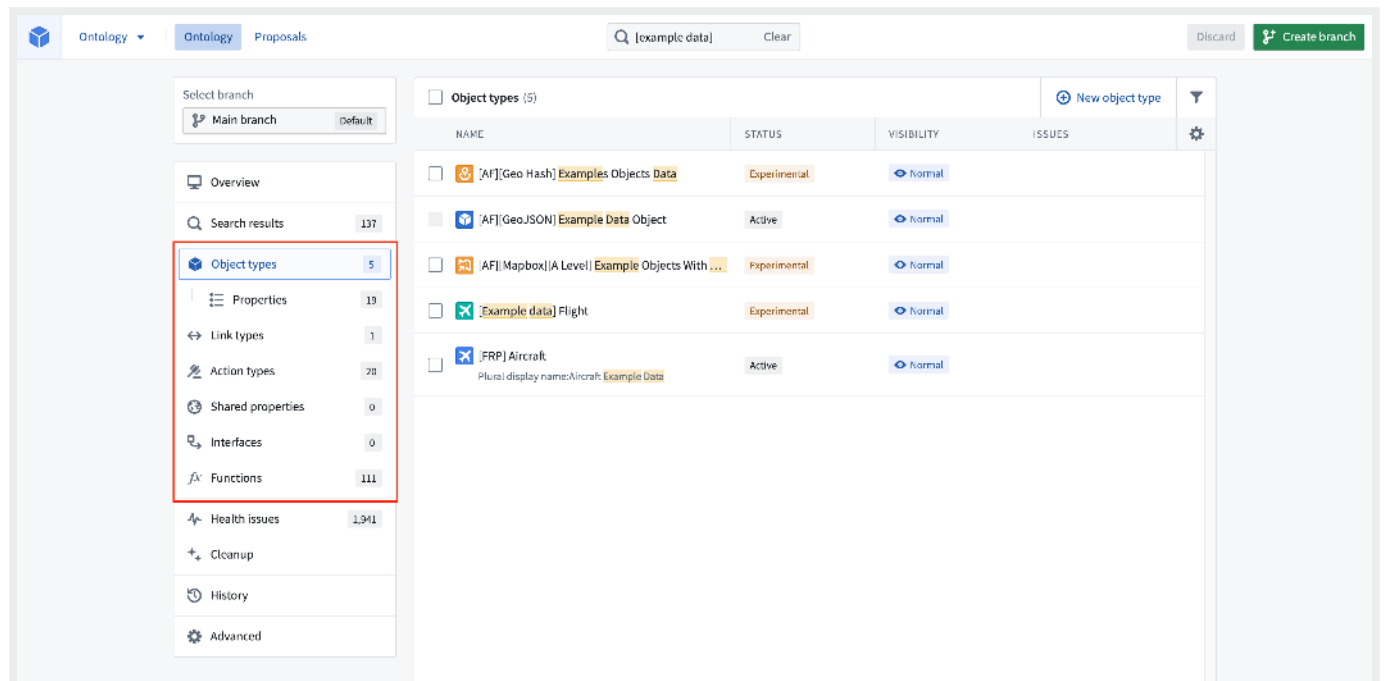


Searching in the **Search** bar from the home page will update the home page with the list of resources that match the search. You can search for any object type, property, link type, action type, shared properties, interfaces, or functions you are interested in. The search results will highlight which field your search term matched on. You can use the up-arrow and down-arrow keys on your keyboard to move through search results and see previews for the selected results. You can select **Open** or use the Enter key to open the selected result.



Home page filters

The **Object types**, **Link types**, **Action Types**, **Shared Properties**, **Interfaces**, and **Functions** pages can be selected from the home page sidebar. These pages allow for filtering object types and link types based on their visibility, development status, and indexing issues.



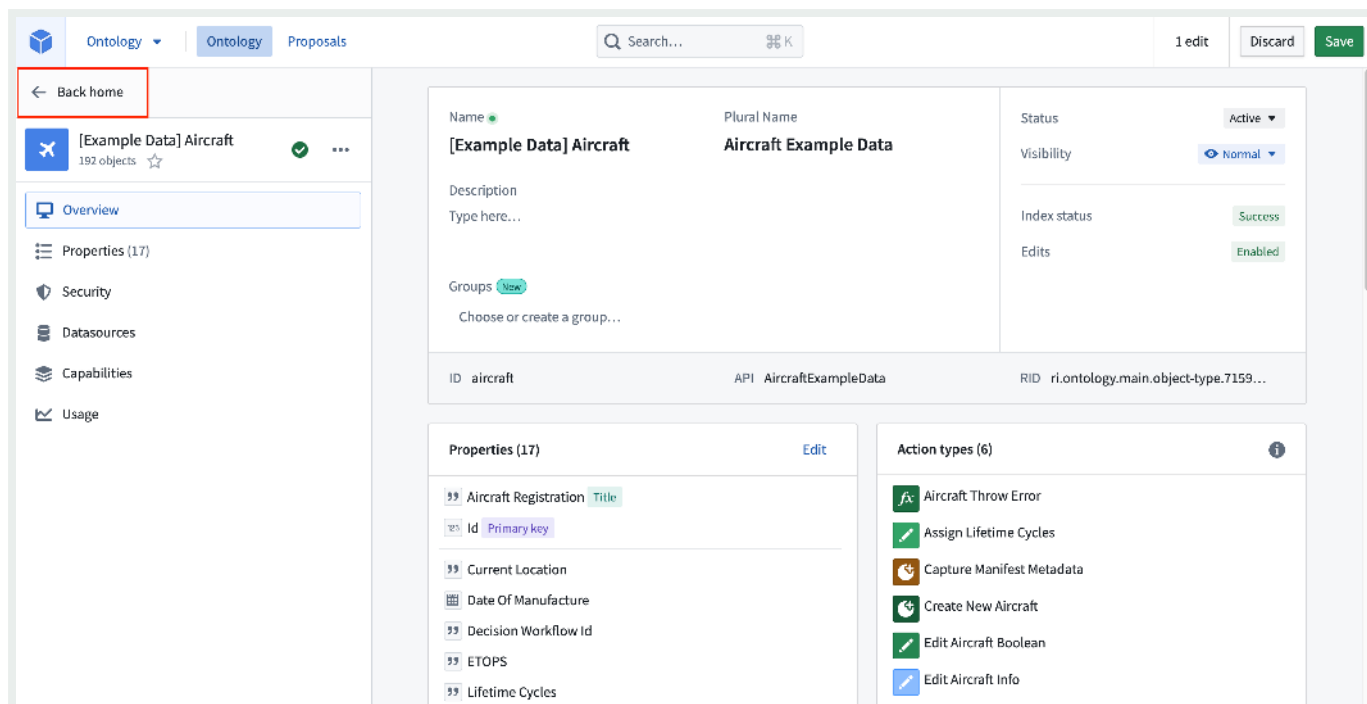
Object types whose backing datasources are unregistered or have failed to reindex into Object Storage v1 (Phonograph) will have red error messages in the issue column of the object type page.

Navigation back from a selection

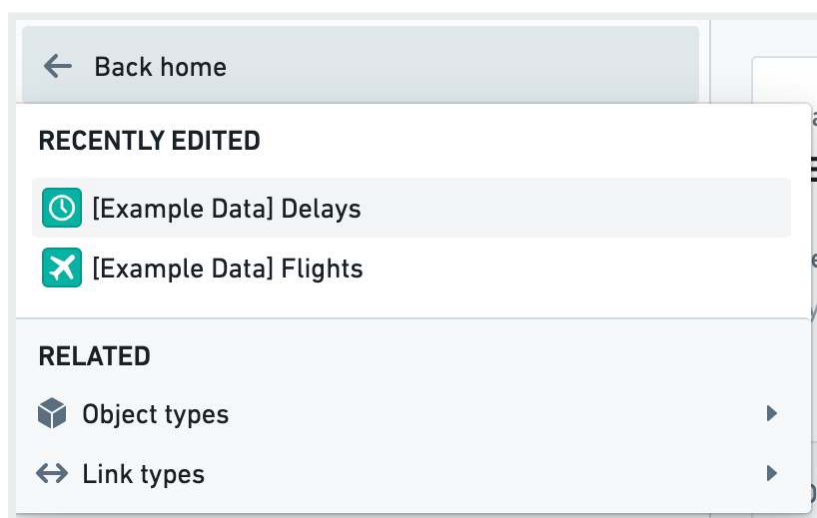
Once you've opened an object type, link type, or action type, you have the option to select **Back home** from the top left corner of the view's sidebar.

[API Reference ↗](#)

[Send feedback](#)



Hovering over the **Back home** button will also bring up quick links to recently edited object types, link types, and action types, as well as all resources that are related to the one you are currently viewing.



PREVIOUS
← Overview

NEXT
Viewing usage →

[API Reference ↗](#)

© 2020 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Send feedback](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Ontology Manager / Viewing usage

Ontology metrics

The Ontology Manager can be configured to show usage metrics for object types and link types.

Key terminology

- **Reads:** A read is recorded when an application loads objects for a specified object type. This can include displaying objects in a table in Workshop, returning all objects from search for a given object type, aggregating a property on an object type, and so on. Note that one read represents one load request from [Object Storage v1 \(Phonograph\)](#) or the Object Set Service (OSS). Many objects loaded or aggregated at once will only be recorded as a single read. Also note that any object type or link type usage happening in Ontology Manager is not included.
- **Writes:** A write is recorded when an application makes edits to objects of this type as the result of an [Action](#), [Function](#), Foundry Form, direct Object Explorer edit, or API call. Note that one write represents one edit request sent to [Object Storage v1 \(Phonograph\)](#). Many objects edited in bulk at once will only be recorded as a single write.
- **Interactions:** The total number of reads and writes on objects of this type over the last 30 days.
- **Active users:** The number of unique user IDs that triggered the reads and writes recorded over the last 30 days.

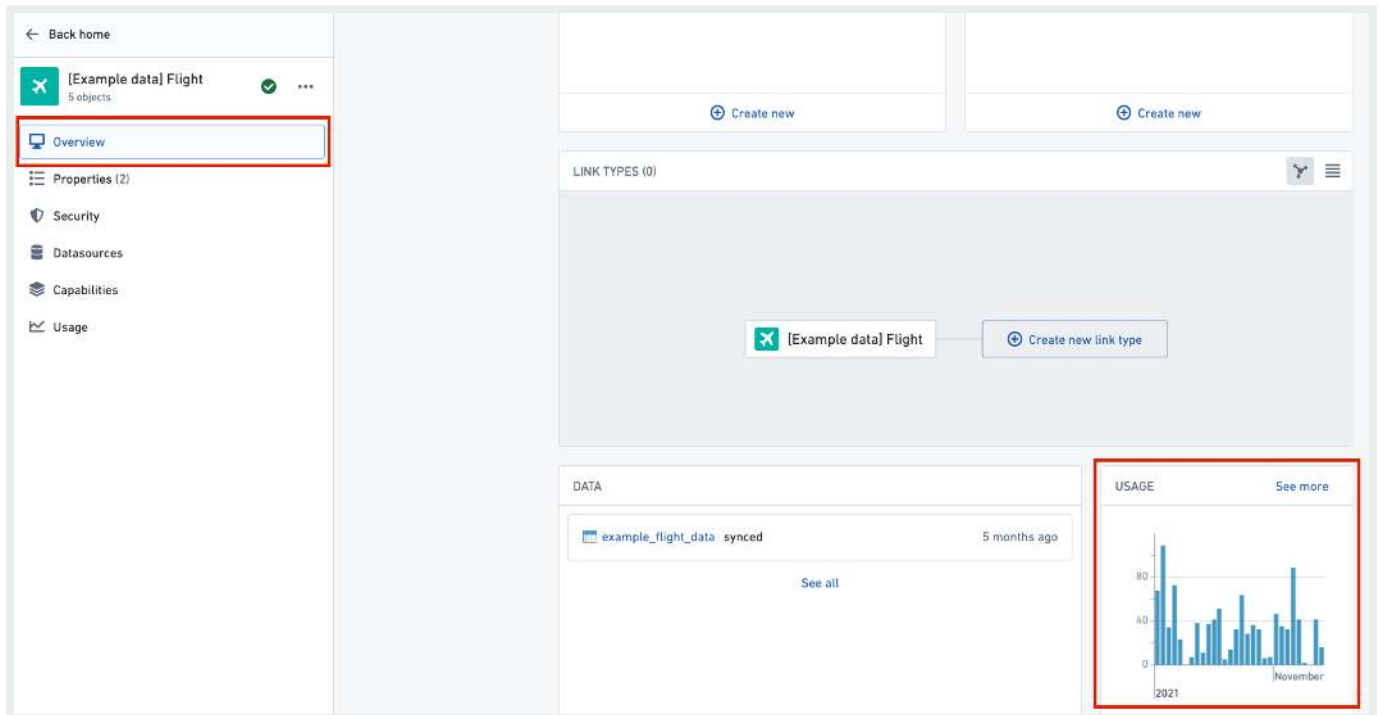
Viewing usage

Tap [API Reference ↗](#) in the Ontology Manager to view object type and link type usage:



Send feedback

- A usage graph on the **Overview** tab: High-level summary of usage over the last 30 days, enabling Ontology users to quickly understand the implications of making a breaking change to this resource.



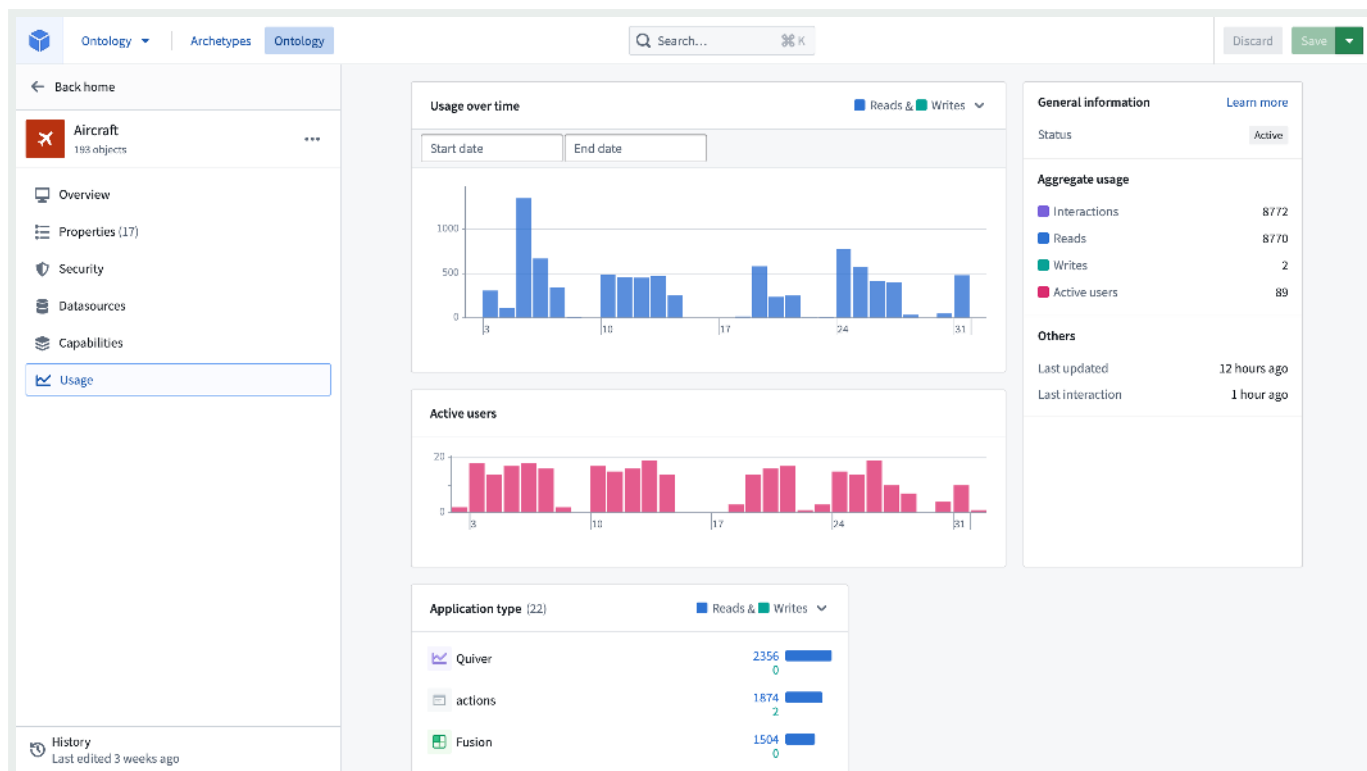
⚠ Warning

If you see “No usage for the last 30 days” in the usage graph when you would expect to see usage statistics, then it’s possible that internal tables may not have been configured. Contact your Palantir representative for more information.

- A dedicated **Usage** tab: Detailed usage metrics for resources. Users can see, over the last 30 days, who has used each object type, when, and in which Foundry applications. The feature is intended to help Ontology users make Ontology changes more safely by providing a clearer understanding of a change's impact. The **Usage** tab can also be accessed by clicking **See more** on the usage graph in the **Overview** tab.

[API Reference ↗](#)

[Send feedback](#)



Enabling Ontology usage

Usage on the **Overview** tab and detailed usage metrics in the **Usage** tab are configured from the **Ontology settings** tab in Control Panel using the **Ontology metrics** toggle. This toggle can only be enabled or disabled by Ontology administrators and changes may take up to 60 minutes to take effect in Ontology Manager.

Shared Ontology usage

If your organization shares an Ontology with another organization, then the **Usage** tab will be accessible by users of all organizations that have the Ontology metrics turned on. The usage metrics displayed only includes the usage from users who have access to the object type and those who are from organizations that have the Ontology metrics enabled. See the steps outlined in [Enabling Ontology usage](#) for more information.

← API Reference ↗

NEXT →
Migrate to project-based permissions

Send feedback

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Migrate to project-based permissions

Ontology resources, including object types, action types, link types, interfaces, and shared properties, can be saved within specific projects and automatically inherit permissions from those projects. Object and link instance permissions remain dependent on the backing datasource location. Once migrated permissions to view, edit, and manage ontology resources are managed through [Compass](#), the Palantir platform's filesystem. [Project-based permissions](#) replaces the previous ontology roles and datasource-derived permissions models. This is the same permission model used for all other resource types.

You can migrate your existing ontology resources to [project-based permissions](#) using our migration tool. This tool suggests the placement of ontology resources into appropriate projects while ensuring they receive the correct permissions.

⚠ Once a resource has been migrated to project-based permissions, it cannot be reverted to ontology roles or datasource-derived permissions.

To turn on project-permissioning for new ontology resources, ontology owners can navigate to the **Ontology configuration** tab in Ontology Manager and toggle on **Require new ontology resources be saved in a project**. Once enabled, users will be prompted to choose a save location when creating new ontology resources.

API Reference ↗

Being aware of the following limitations:

Send feedback

- This feature is not yet available for Default ontologies. Contact Palantir Support if you are not sure of your ontology type.
- Ontology resource names must conform to Compass conventions. Forward slashes ("/") are not allowed, and duplicate names are not permitted. While aliases allow duplicate names to be rendered, the system removes duplicates by appending "(1)" to ensure unique paths.
 - Each ontology resource must have a unique name
 - Example: `common/utility-room` is invalid due to the forward slash
- An Ontology's resources must be saved in a project within the same space as the ontology itself.

Approaches to migration

Before starting the migration, consider how you want to organize your ontology resources:

- **Save ontology resources alongside datasources or in use case projects:** Keeping ontology resources next to their corresponding datasources ensures consistent permissions across resources and instances. This approach lets you grant permissions to the entire use case in one place, ensuring the right users can view, edit, or manage all components together.
- **Save ontology resources in a dedicated project:** Create one or more separate projects specifically for ontology resources. Grant broad access to these projects to make ontology resources viewable to everyone who needs them.
- **Hybrid approach:** Save core ontology resources into a single project that everyone has permissions to view. Save use case specific resources in use case specific projects. This prevents ontology pickers and search screens becoming cluttered with use case specific ontology resources.

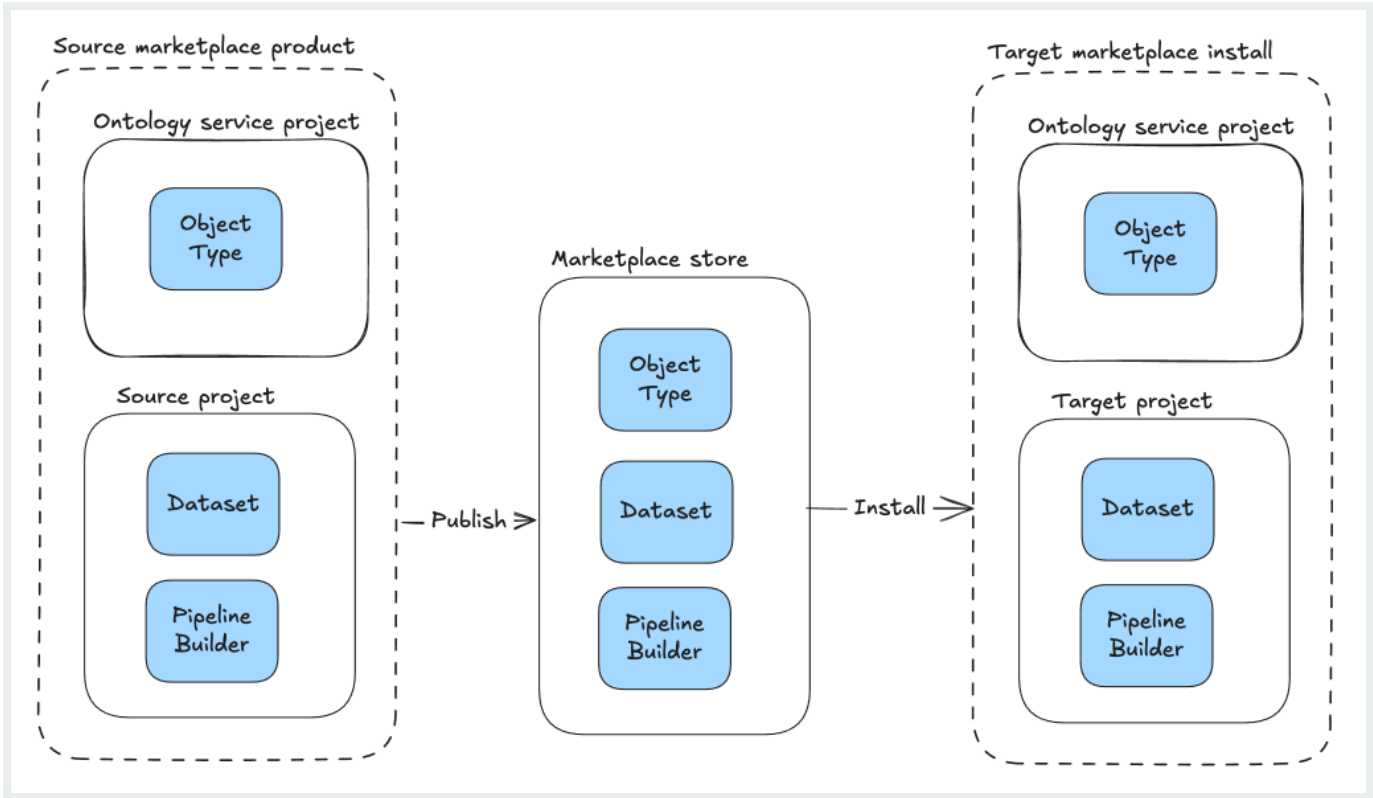
i Ontology resources have separate permissions from object and link instances. This migration affects only ontology resource permissions. Object and link instances permissions remain based on the backing datasource location.

[API Reference ↗](#)

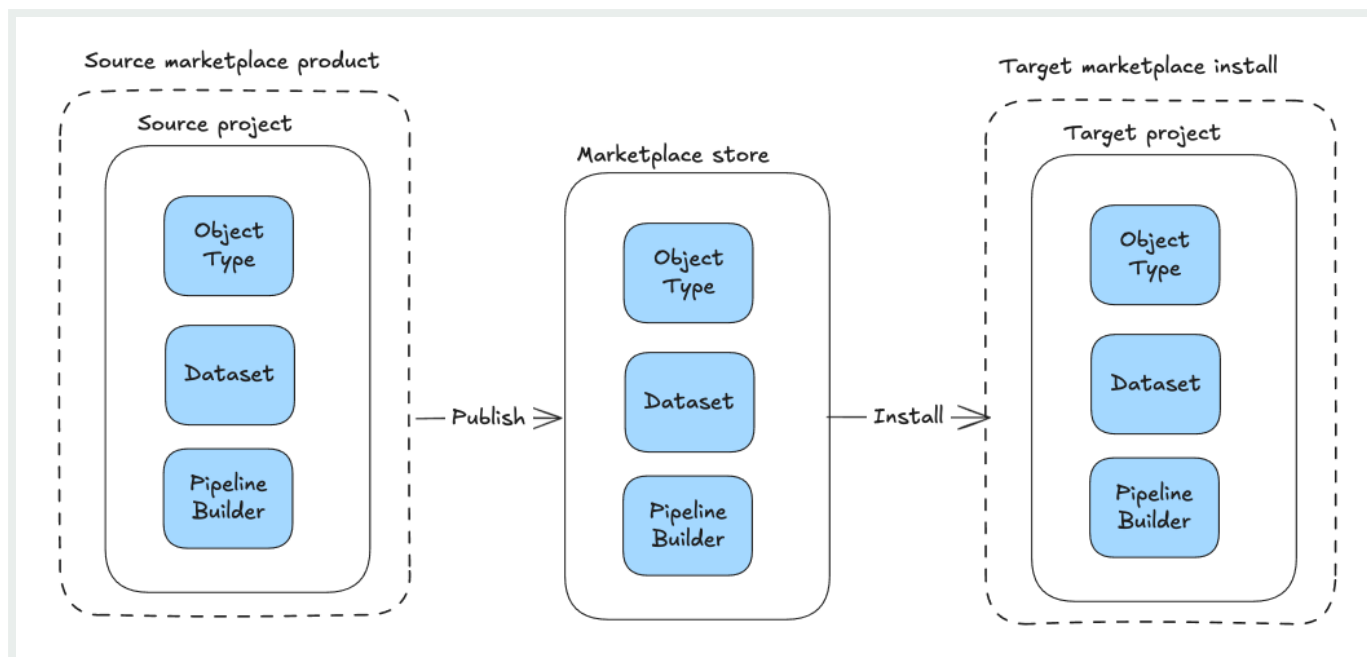
How migration changes Marketplace installs

[Send feedback](#)

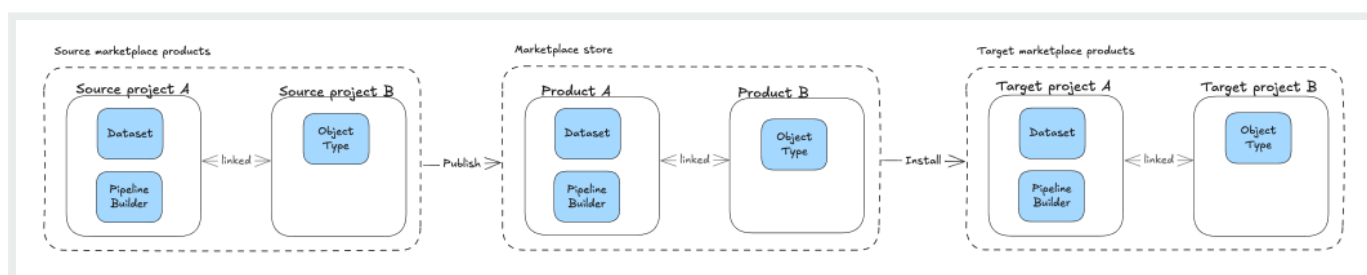
Before you migrate to project permissioning, ontology resources live in the **Ontology service project**, a system-managed project that holds all ontology resources for an ontology under the legacy permission models. In the Ontology service project, every user has a default **Viewer** grant on ontology resources, and the resources do not carry file classifications. When you install a Marketplace product, the ontology resources are placed in this Ontology service project, while the non-ontology files are placed in the target project the installing user chooses:



After you migrate to project permissioning, the same product installs the ontology resources directly into the chosen target project alongside the rest of the files, and the project's role grants and classifications apply to them:



The Ontology service project gives every user a default Viewer grant and carries no classifications, so the target project's tighter role grants — along with any mandatory markings or maximum classification it enforces — can narrow the visibility of installed ontology resources compared to before migration. To keep the ontology resources at a different visibility from the other files in the product, split the product into two linked Marketplace products: one containing the ontology resources, installed into a more permissive project, and a second containing the remaining files, installed into a more restricted project.



Use the migration assistant (recommended)

The migration assistant helps you quickly identify suitable projects and locations for your ontology resources.

To [API Reference](#) **Migration assistant:** Select your ontology, navigate to the **Ontology configuration** page, and select **Proceed to migration** under the **Migrate** tab.

[Send feedback](#)

Migrations

Project permission migration

Migrate resources to project permissions based on generated recommendations

Proceed to migration

Strong recommendations for where to move resources are preselected to accelerate your workflow, while weaker suggestions remain unselected for your review. After confirming your selections, proceed with the migration. Before finalizing, you can create necessary imports or cancel the operation.

← Ontology configuration

Ontology projects migration assistant

Overview

Recommended projects1

procurement inputs15

Individual resources146

← All projects

Resources recommended for procurement inputs

Choose from resources you have permission to migrate

procurement inputs15

Name	Rationale	Status	Type	Groups	
[Proc] Department	Recommended	Experimental	Object type		
[Proc] Source Category	Recommended	Experimental	Object type		
[Proc] Purchase Order	Recommended	Experimental	Object type		
[Proc] Goods Receipt Item	[Proc] Employee	Recommended	Object type		
[Proc] Location			Object type		
[Proc] Purchase Order Item	placeholder-employee	Recommended	Object type		
[Proc] Employee		Recommended	Object type		
Parent Source Category	Child Source Category	Recommended	Link type		
Buyer	Purchase Order	Recommended	Link type		
Vendor	Purchase Order	Recommended	Link type		

This project has been recommended as it includes the datasource which backs this object type.

placeholder-employee

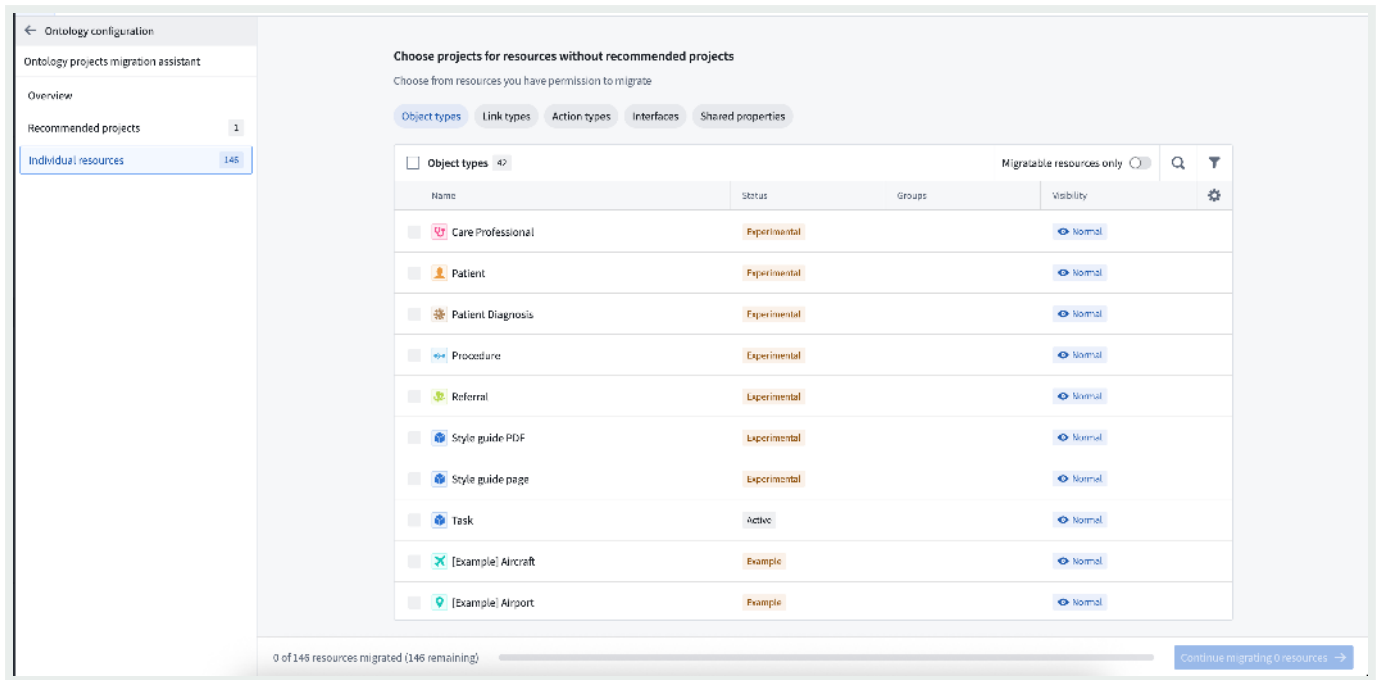
procurement inputs

0 of 146 resources migrated (146 remaining)

View project permissions

Continue migrating 0 resources→

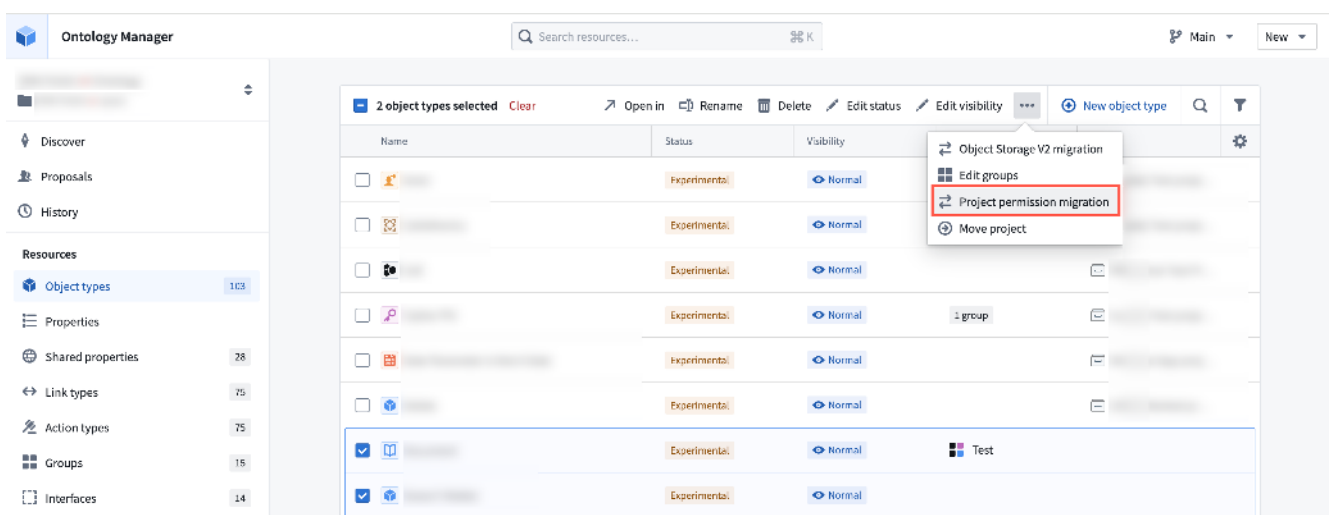
These recommendations help you make faster, more informed decisions about resource placement. If no recommendations are available, you can manually select locations in the **Individual resources** tab of the migration assistant.



Migrate resources directly

You can also migrate resources without using the assistant, which is useful when you know exactly where resources should go or want to migrate specific resources quickly.

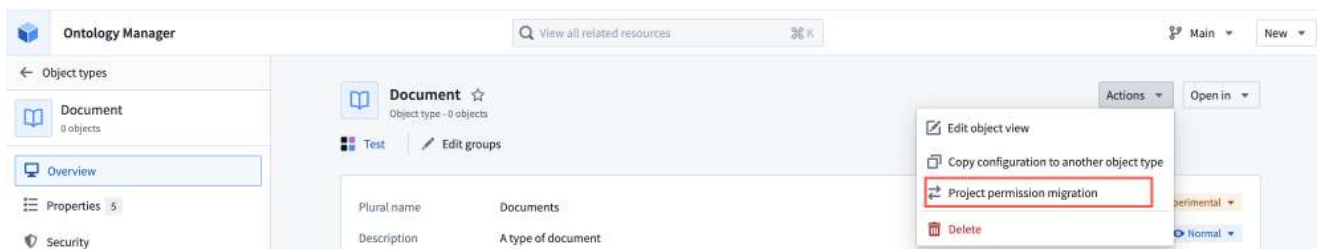
- **Bulk migrate multiple resources:** Select your ontology, then choose a resource type from the **Resources** section in the left sidebar. Select the items to migrate, then use the dropdown menu to select **Project permission migration**.



API Reference ↗

Individual resource: Open an ontology resource and use the **Actions** dropdown menu on the **Overview** page to select **Project permission migration**.

Send feedback



PREVIOUS
← Viewing usage

NEXT
Ontology roles migration [Legacy] →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Ontology building / Ontology Manager / Ontology roles migration [Legacy]

Ontology roles migration [Legacy]

Legacy

This page describes a legacy process that is no longer the most up-to-date method for ontology resource permissioning. Ontology resources can now be [permissioned using the Compass filesystem](#). To migrate over, [review the migration guide](#).

[Ontology roles](#) is a legacy authorization model for ontology resources, replacing [datasource-derived permissions](#). Ontology roles apply to object types, link types, and action types, which are referred to as “ontology resources”. Ontology roles are required to use the most recent features of the ontology, including shared property types, interfaces, and more. By default, ontology roles have been used on all new enrollments created since September 2023.

Ontology roles allow you to grant roles *directly* on each Ontology resource and to their metadata. All metadata permissions are managed in Ontology Manager. Ontology roles allow you to decouple Ontology resources (like an `Aircraft` object type) from the *data* of these resources, such as [object instances](#) (like a specific plane represented as an Ontology object) and link instances; these data instances remain governed by their input datasource permissions. Learn more about [Ontology permissions](#).

Each Ontology is linked to one or multiple [Foundry organizations](#).

Roles permissions are detailed in the [Ontology roles documentation](#).

API Reference ↗

Object type, link type, and action type migrated to ontology roles will have view permission for all users that have access to that particular ontology. Ontology resource information is accessible for all users of that ontology.

Send feedback

There is no explicit ordering required for migrating ontology resources to ontology roles, making it possible to have a mix of object types using [datasource-derived permissions](#), and others using ontology roles. Therefore, for the purpose of this migration, we define `able to edit an object type` as follows:

- **Object types using datasource-derived permissions:** Users have `Editor` permission on the input datasources for that object type.
- **Object types using Ontology roles:** Users have `Ontology Editor` permission on the object type and do not require any permissions on the input datasource. Users should note that this only allows editing Ontology resources and their metadata and does not grant any permission on the data/datasource itself. Access to object data (not metadata) is still governed by the permissions granted on input datasources.

Migrate to ontology roles

If Ontology roles are enabled on your enrollment, but you have not yet migrated the Ontology resources that you own to the roles authorization model, you will see a banner in Ontology Manager that prompts you to do so. Alternatively, when looking at the tables of Ontology resources, you can filter to the ones you own that are still using the "Same permissions as backing datasource".

i Ontology roles are not yet available for all Foundry enrollments. If you do not have access to Ontology roles, contact Palantir Support.

Prerequisites

To perform the migration, users must have the following permissions:

1. Permissions to make changes in the Foundry Ontology.
2. The following permissions for each individual resource:
 - For object types, users must have `Owner` permission on the input datasource.

[API Reference](#) ↗ any link types, users do not need additional permissions. Only `Viewer` permissions on both object types referenced in the link type are required, and the user must be an `Owner` on the link type itself or at the Ontology

[Send feedback](#)

- For many-to-many link types, users must also have **Owner** permission on the input datasource of the link type.
- No additional permission is needed for action types.

i Additional permissions are required for [modifying action types](#) after migrating to Ontology roles. You will need permissions on any object types the action type edits, and on the action type itself, to modify action types with roles.

Notification to migrate

Migration to Ontology roles is managed through [Upgrade Assistant](#). A dedicated Upgrade Assistant task displays the list of Ontology resources that must be migrated, along with assignees that have permission to perform the migration. Opening a resource from Upgrade Assistant directs the user to the **Security** tab of the resource to migrate it within the Ontology Manager application.

If a user has permission to set Ontology roles, Ontology resources that have not yet migrated will appear in the migration interface in the **Security** tab to set Ontology roles for the first time. After the Ontology resource is migrated, the Ontology role picker is displayed under the **Security** tab and can be updated by **Ontology Owners**.

Migrate using Ontology Manager

There are two ways to migrate Ontology resources (object types, action types, link types) to Roles: [bulk migration](#) of Ontology resources or [one-by-one migration](#) for all Ontology resources.

Bulk migration of Ontology resources

You can bulk-migrate Ontology resources to Roles. Refer to the list of [prerequisites](#) before proceeding. You can only migrate 500 resources at a time.

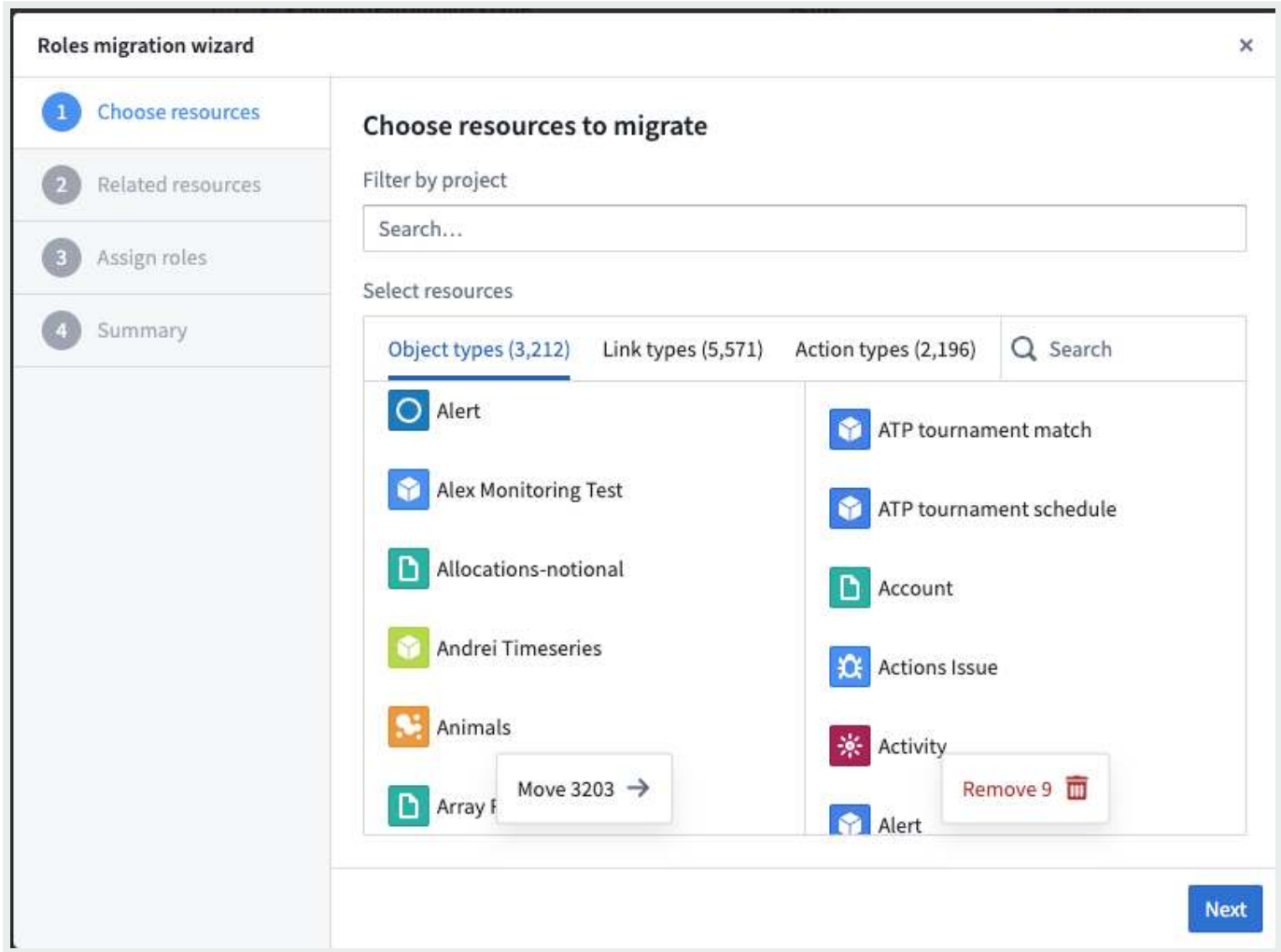
API Reference ↗

To **continue with migration**, select **Advanced** in the Ontology Manager and select **continue with migration assistant** in the Migrations section.

Send feedback

The migration wizard will appear with the following steps.

- 1. **Choose resources:** Choose all the object types, link types, or action types on which you wish to apply the same roles. The image below displays the interface for choosing resources in the migration wizard:



- 2. **Related resources:** This step is shown only if there are related resources (link types or action types) that can be migrated.
- 3. **Assign roles:** You can manually set the groups/users and the default role you want to assign to these Object types. The roles migrator cannot suggest roles in bulk for now. We strongly recommend using **user groups** over **users** when granting roles.
- 4. **Summary:** View the summary, acknowledge the implications of the migration, and complete the migration to apply the roles.

API Reference ↗

nowledge the implications of the migration, you are acknowledging
that the permissions previously derived from backing datasour

Send feedback

overwritten by the newly assigned role grants and that the migration is irreversible.

Exceptions to bulk migration

Some action types cannot be bulk-migrated without further steps.

Action types cannot be migrated to roles if:

- The action type has no submission criteria that checks the current user's permissions.
 - Solution: Add a condition based on the current user in the **Security & Submission criteria** tab of the Action type in the Ontology Manager (in the **Submission criteria** section) and save to the Ontology before re-attempting the migration.
- The action type is backed by a function and must be republished with `@edits decorators` before the migration.
 - Solution: Refer to the steps in the [decorators](#) documentation.

To address these issues, go to the **Action Types** page and filter down to **Same permissions as backing datasource**, which will display the action types that have not been migrated because of issues. Resolve all outstanding issues and then proceed with bulk migration again for any remaining action types.

Warning

When migrating action types, ensure that permissions are only granted to users with sufficient context on both the referenced object types and all use cases where the action type is deployed.

One-by-one migration for all Ontology resources

From the Ontology Manager home page, select the Ontology resources that you want to migrate (object type, link type, or action type) and filter by `Security setup to Same permissions as backing datasource` and by `permissions = owner`.

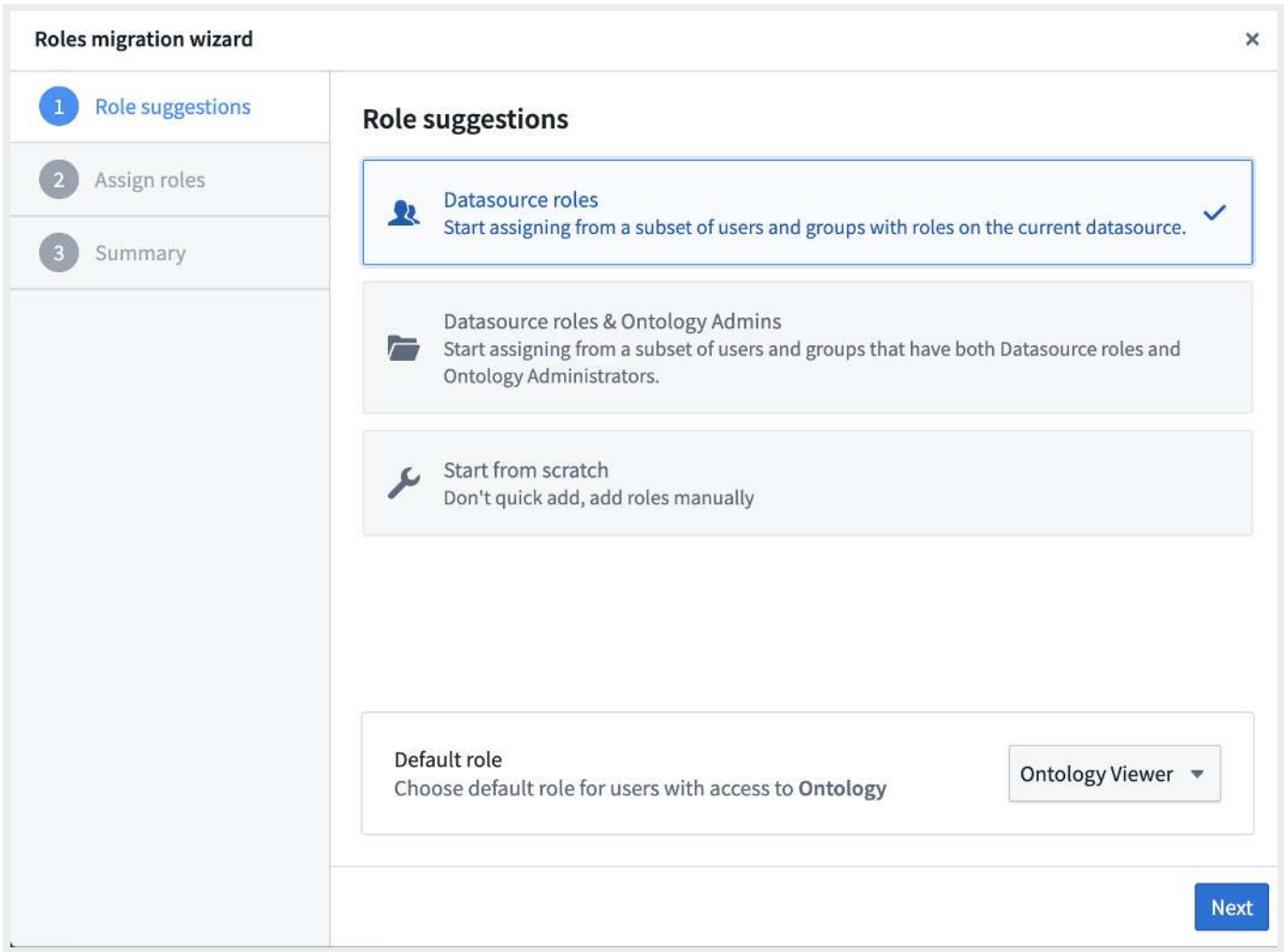
See [API Reference](#) to migrate, then choose either **Migrate to roles** on the header or **Start using roles** on the Security tab.

Send feedback

The migration wizard will appear with two suggested role options: **Datasource roles** and **Ontology admins and datasource roles**. These options allow users to use and configure roles that align with their existing Ontology setups. Selecting one of the options pre-fills the list of suggested roles; these roles can be changed later.

- **Datasource roles:** This option adds all users and user groups with an `Editor` or `Owner` role on the input datasource.
- **Ontology admins and datasource roles:** This option adds all users that can currently modify the given Ontology resource based on [datasource-derived permissions](#). These users belong to the `Ontology Administrators` user group and have an Editor or Owner role on the input datasource.

The image below shows the roles suggestion in the migration wizard:



On the `API Reference` page, review the suggested [Ontology roles](#) and the default role. Then, make modifications if necessary. Next, view the summary, acknowledge the implication, and complete the migration to apply the changes. [Send feedback](#)

The image below shows how to assign roles in the migration wizard:

The screenshot shows a 'Roles migration wizard' window with a sidebar on the left containing four steps: 1 Role suggestions, 2 Assign roles (highlighted), 3 Related resources, and 4 Summary. The main area is titled 'Assign roles' and features a search bar 'Add a user or group...'. Below the search bar, a list shows a user 'john' assigned the role 'Ontology Owner'. A 'You' user is also listed with a '+ Add' button next to it. At the bottom, there is a 'Default role' section with the text 'Choose default role for users with access to **Ontology**' and a dropdown menu set to 'Ontology Viewer'. 'Back' and 'Next' buttons are located at the bottom right.

Exceptions

You may be prevented from attempting the migration of an action type to Roles if:

- The action type has no submission criteria that checks the current user's permissions.
 - Solution: Add a condition based on the current user in the **Security & Submission criteria** tab of the action type in the Ontology Manager (in the **Submission criteria** section) and save to the Ontology before re-attempting the migration.
- The action type is backed by a function and must be republished with `@edits` decorators before the migration.
 - Solution: Refer to the steps in the [decorators](#) documentation.

When [API Reference](#) have been resolved, you will be able to migrate the action types to Roles.

What if some ontology resources are no longer needed

[Send feedback](#)

If you no longer need an ontology resource that requires an action, you can delete the resource using the [Ontology cleanup tool](#). This tool helps you identify which object types are safe to delete based on a set of criteria (for example, if the datasource is trashed, the deprecation date has passed, the index is failing, whether they are using Ontology roles or not, and so on.)

Migrating function-backed action types

Function-backed action types require the backing Function to be migrated. Foundry will attempt to migrate Functions automatically, allowing you to migrate the action type as well. Some Functions, however, might not be migrated automatically and require manually declaration of the edited object types in the Functions. Check the [FAQ](#) section if you see failures in your Function-backed actions after migrating an action type to Ontology roles.

Migration frequently asked questions

Will there be any changes in object-aware applications?

Users may have access to metadata on object types and link types in the [Ontology Manager](#) that they could not previously view because they did not have `Viewer` access to the input datasources. This does not mean they will get access to the data in the input datasources which is always governed by the roles on those datasources. The specifics of what metadata they will be able to see is determined by their [Ontology role](#).

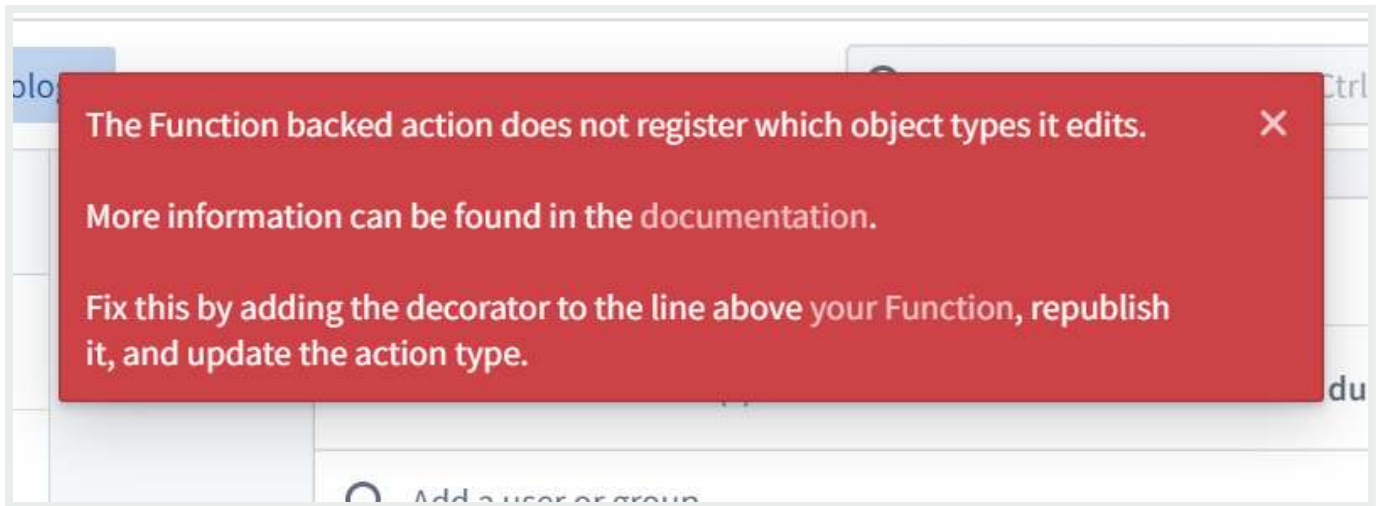
Object Explorer will also update the permission checks for editing Object Views when an object type is migrated. Before an object type is migrated, a user needs to be in the `Object View Administrators` group and have `Editor` access to the input datasources to edit Object Views. After an object type has been migrated, Object Explorer will only check that the user has `Ontology Editor` permissions on the object type itself.

How can I resolve a function-backed action that is failing post migration?

If Foundry failed to automatically detect the object types an Ontology Edit Function is editing, the action will fail after submission with the following error.

API Reference ↗

Send feedback



To resolve this error, follow the steps explained in the [Functions documentation](#).

PREVIOUS
← Migrate to project-based permissions

NEXT
Change management / Save changes to the Ontology →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

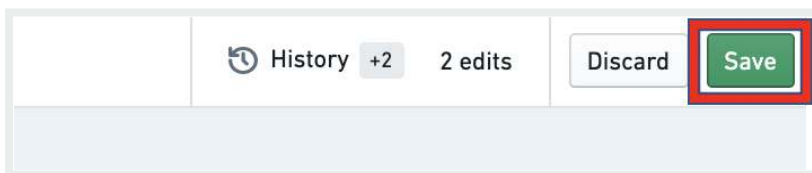
Ontology building / Ontology Manager / Change management / Save changes to the Ontology

Save changes to the Ontology

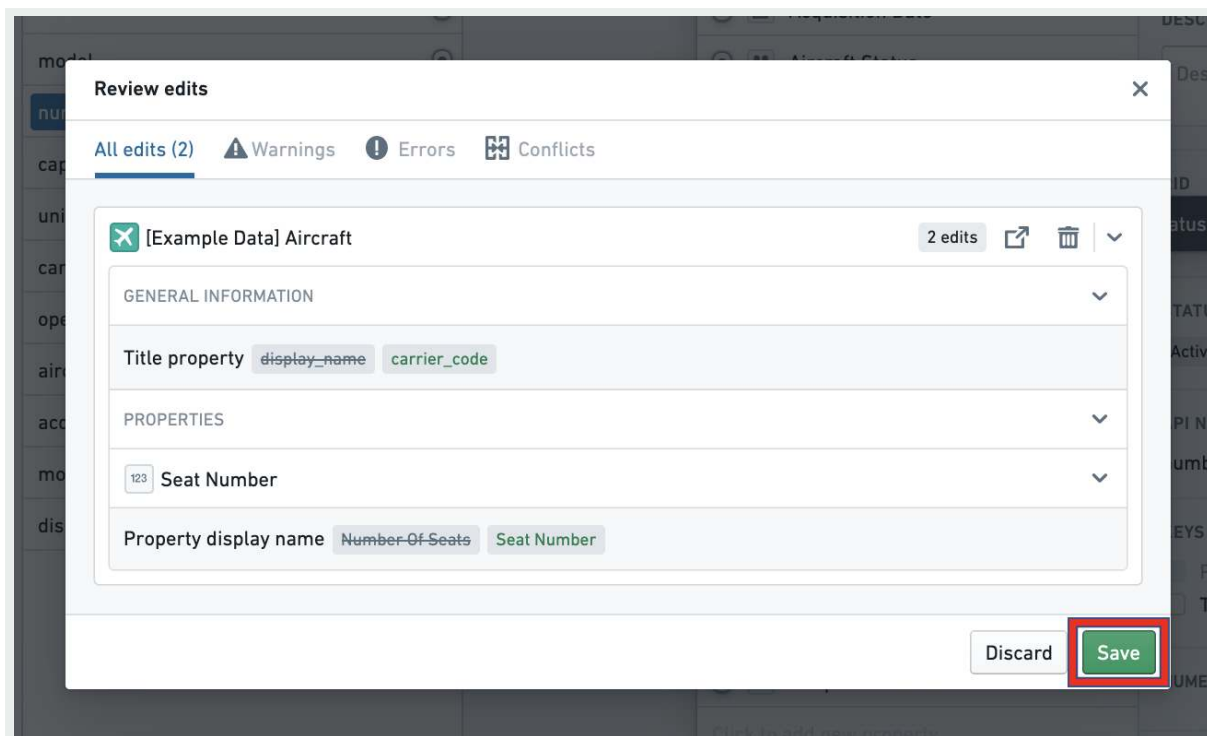
Save your changes

Any changes you make in the Ontology Manager are stored locally in a work-in-progress state. For these Ontology changes to be available for others and reflected in user-facing applications, you must save your changes. To save changes:

1. Select **Save** from the Application header at the top-right corner of the application.



2. Open the **Review edits** dialog to review all your changes.
3. Finally, select **Save** to update the Ontology.



Handle errors and warnings

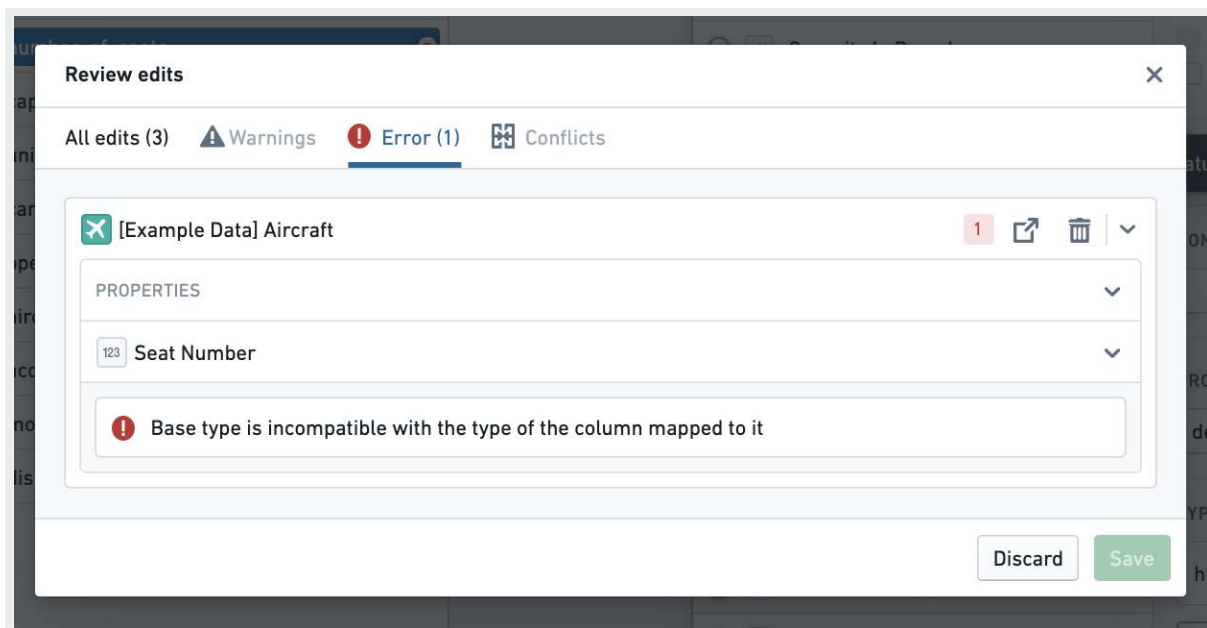
If the **Save** button is grayed out, you may have an error that is stopping you from saving. To resolve this, you can:

- Scroll through your changes and view the error messages in line, or
- Select the **Errors** tab at the top of the **Review edits** dialog to see the errors preventing you from saving.

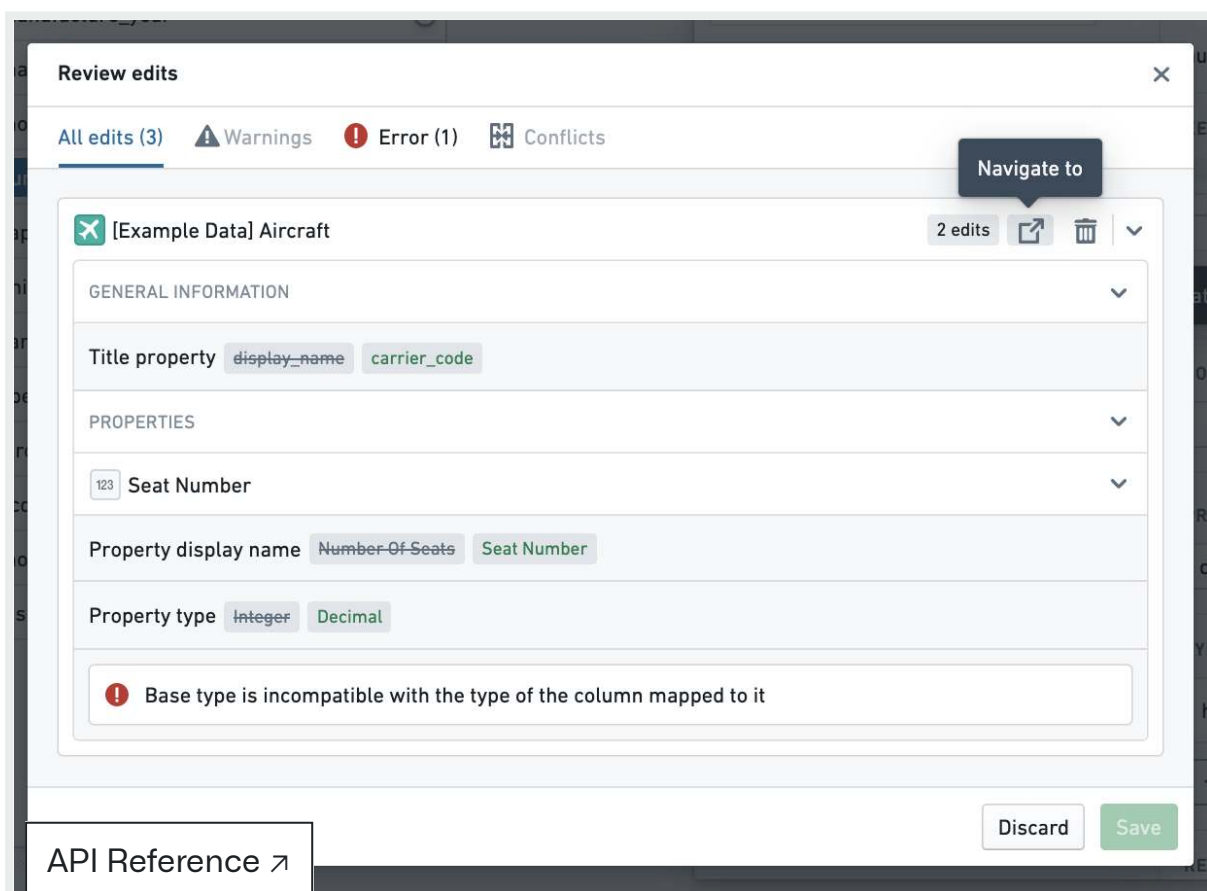
The **Review edits** dialog will also show you warnings in-line and in the **Warnings** tab for changes you are encouraged to make. While errors need to be handled in order to save, warnings will not prevent you from saving.

[API Reference ↗](#)

[Send feedback](#)



If you receive an error, you can use the open shortcut to navigate to a resource you need to edit before saving.



Send feedback

i Changes to Functions can only be made in the Functions repository, and not in the Ontology Manager. You can navigate to the Functions Repository from the Functions Entity view in the Ontology Manager.

Handle updates and merge conflicts

The **Save** button may also be grayed out if the Ontology has been saved by another user since you began making your changes. You will need to select **Update** from the top of the Review edits dialog to merge the other user's changes with your own.

Review edits

There are new updates to the Ontology that you can easily and safely merge with your own.

Update

All edits (1)

Warnings

Errors

Conflicts (0)

Origin Airport 2 ↔ Departing Flight 2...

1 edit

LINK TO DEPARTING FLIGHT 2...

1 edit

Display name

Departing Flight 2

Departing Flight 2...

Discard

Save

It is possible that there are merge conflicts between changes another user has made and the changes in your working state. You will be prompted to resolve them. You can choose between keeping the changes in the latest version of the Ontology or overriding them with the changes in your working state.

[API Reference ↗](#)

[Send feedback](#)

Review edits

1 entity you have edited has been updated since you started working on your changes.

Review

All edits (1)

Warnings

Errors

Conflict (1)

Some updates made to the Ontology since you started editing are on the same entities as yours. For each, choose to either use the latest changes or keep yours.

Aircraft 2

Latest version

GENERAL INFORMATION

1 edit

Display name

Aircraft

Aircraft 1

Use latest

CK Your changes

GENERAL INFORMATION

Display name

Aircraft

Aircraft 2

Keep my changes

Apply

Discard

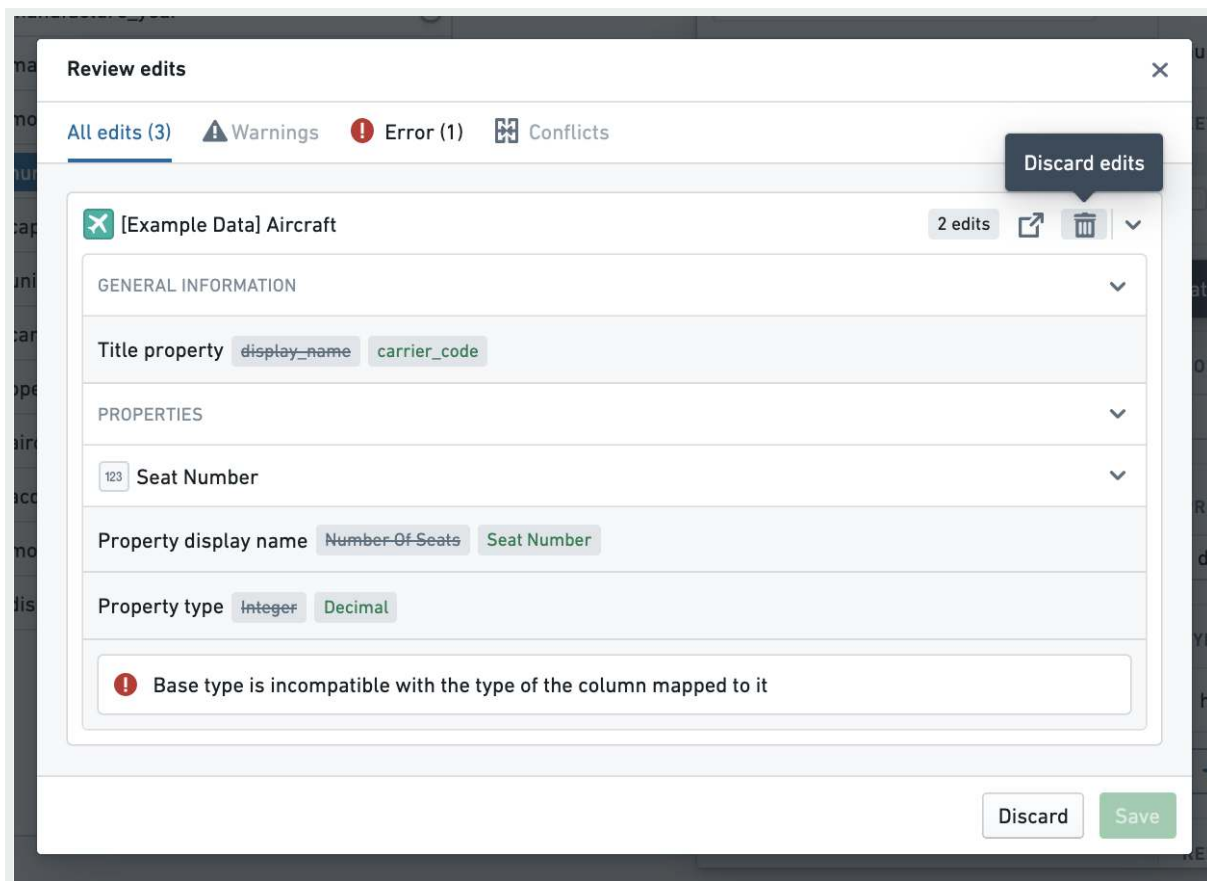
Save

Discard your changes

Each resource in the Ontology that you edit will have its own entry in the **Review edits** dialog. You can discard the changes you made to a resource by hovering over the entry in the **Review edits** dialog and selecting the trash icon.

API Reference ↗

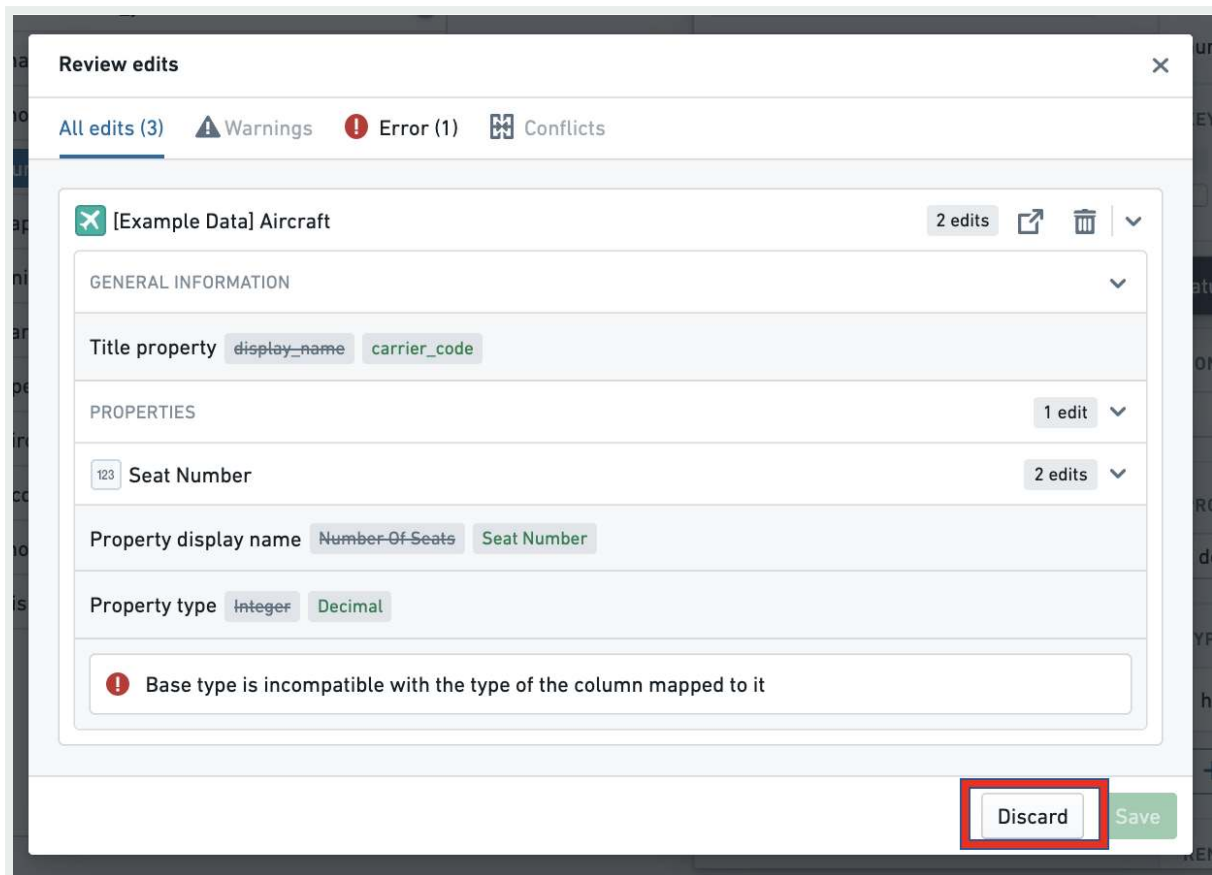
Send feedback



You can discard all unsaved changes you made to the Ontology at any point by selecting the **Discard** button in the header at the top right of the application, or by selecting **Discard** at the bottom of the **Review edits** dialog.

[API Reference ↗](#)

[Send feedback](#)



Respond to a warning message

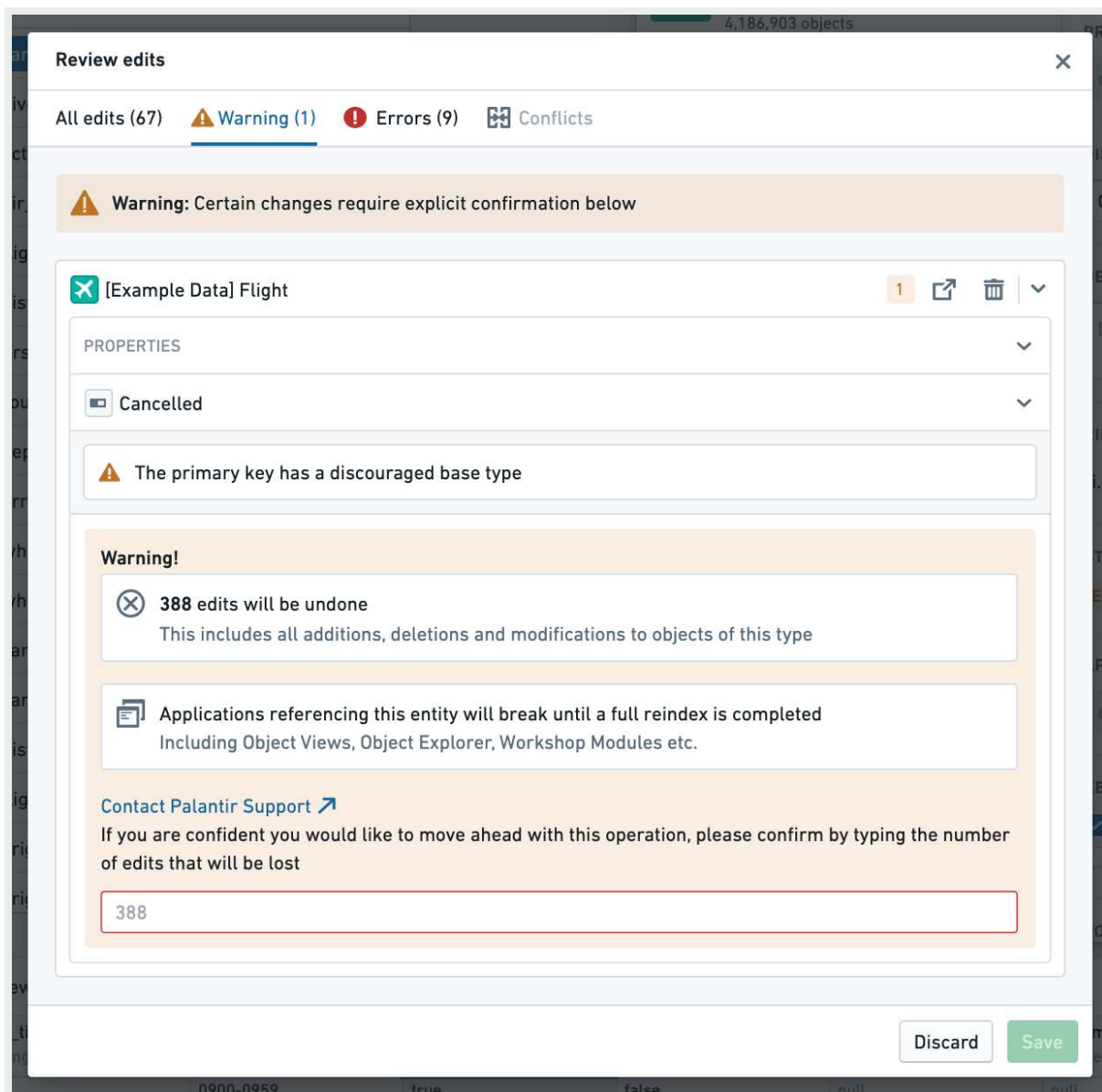
As you review your changes in the **Review edits** dialog, you may get a warning message that prompts you to confirm the warning before saving.

Edits to object types and their properties can have an application-breaking impact on applications relying on those object types. Furthermore, if an object type has writeback enabled, extra caution should be taken when making edits to that object type to ensure that the history of edits made to objects of that type is not removed.

For a full description of which changes can be destructive, read more about [potential breaking changes](#).

Once you have read through the impact of your changes detailed in the warning message and the [API Reference](#) implications of those changes, you can type in the name of the entity you edited to proceed with saving.

Send feedback

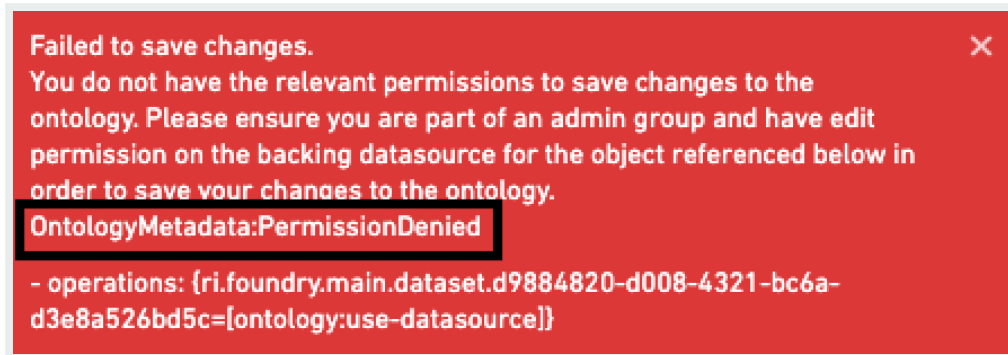


Troubleshooting when a save fails

If the backend services powering the Ontology encounter a problem when you save, you will receive an error message "toast" (pop-up), as in the image below. At the end of the text explaining why you can't save, the name of the error message will be printed. The error message name will begin with the prefix `OntologyMetadata:` or `Phonograph2:`.

[API Reference ↗](#)

[Send feedback](#)



Throughout the Ontology documentation, there are references to the most common errors associated with different changes made to the Ontology. If you see an error message, search for it in the documentation to see if the error and its remediation are documented.

PREVIOUS
← [Ontology roles migration \[Legacy\]](#)

NEXT
[Review and restore changes](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

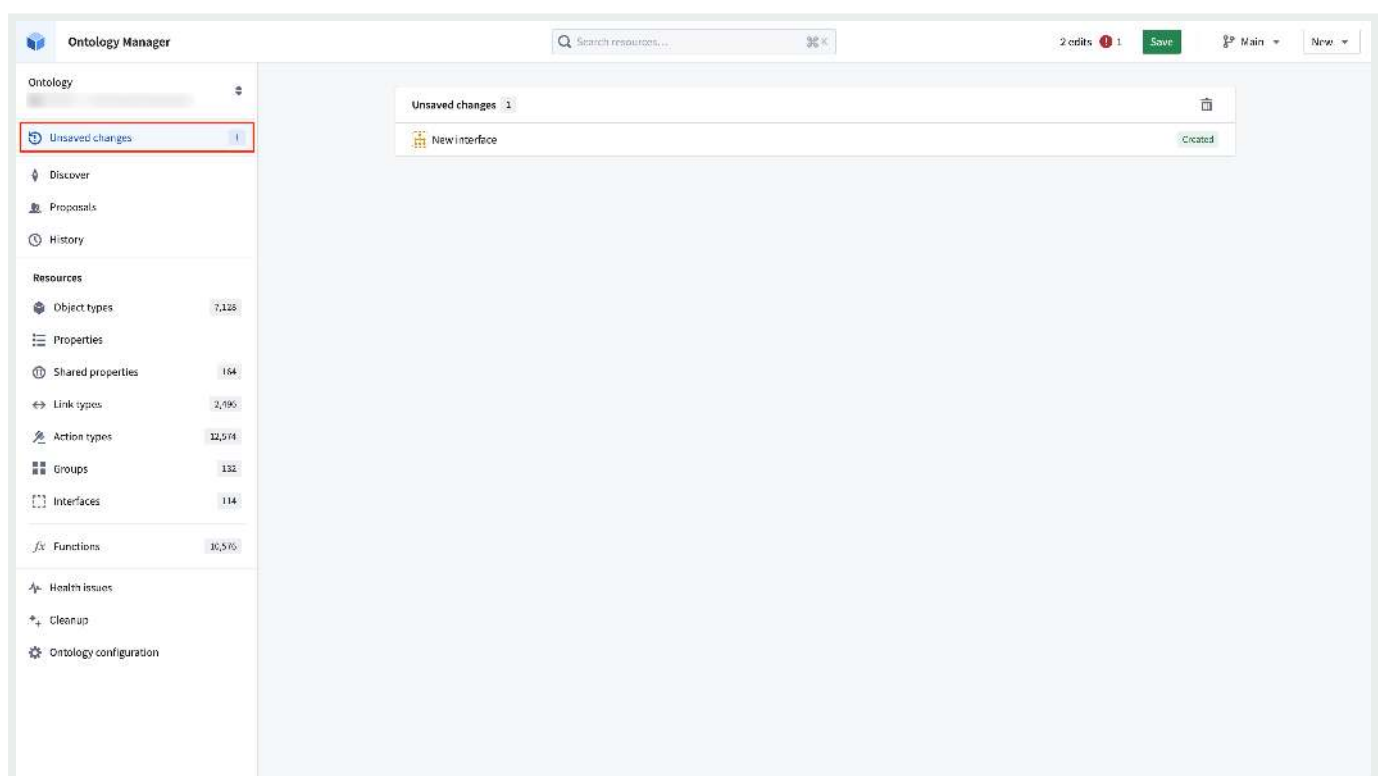
Ontology building / Ontology Manager / Change management / Review and restore changes

Review and restore changes

Review changes made to the Ontology globally

Review unsaved changes made to the Ontology globally:

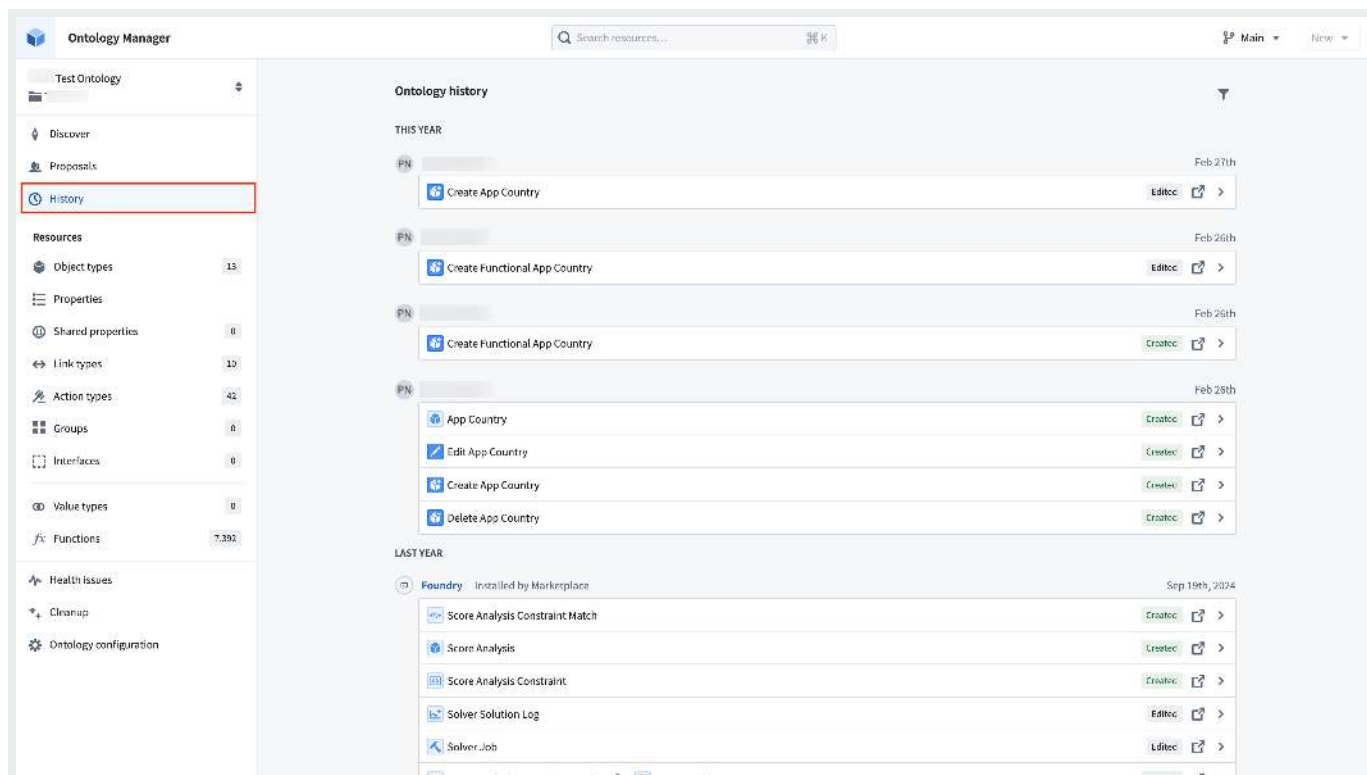
From the homepage sidebar, select **Unsaved changes** to view a list of all unsaved changes made by you.



Review API Reference ↗ changes made to the Ontology globally:

Select the **History** tab in the homepage sidebar to view a list of all saved changes with details on when the changes were made and the user who applied them

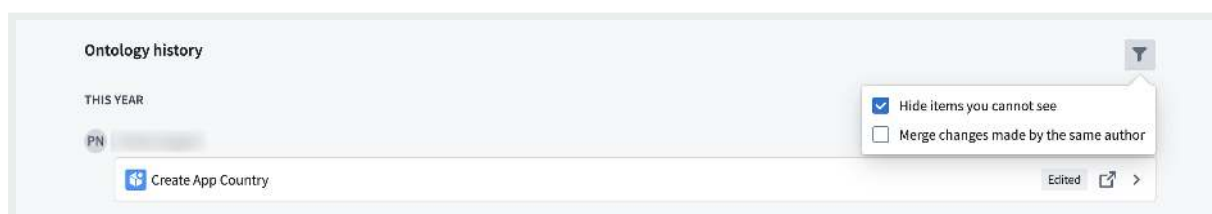
Send feedback



By default, the list of changes are collapsed. You can select the  (down arrow) on any change to view details.

Filter changes made to the Ontology globally

In the **Ontology history** page, you have the option to hide changes to object and link types that you do not have access to view. If this option is not enabled, you will be able to see that a change was made to the Ontology but will not be able to view additional details.



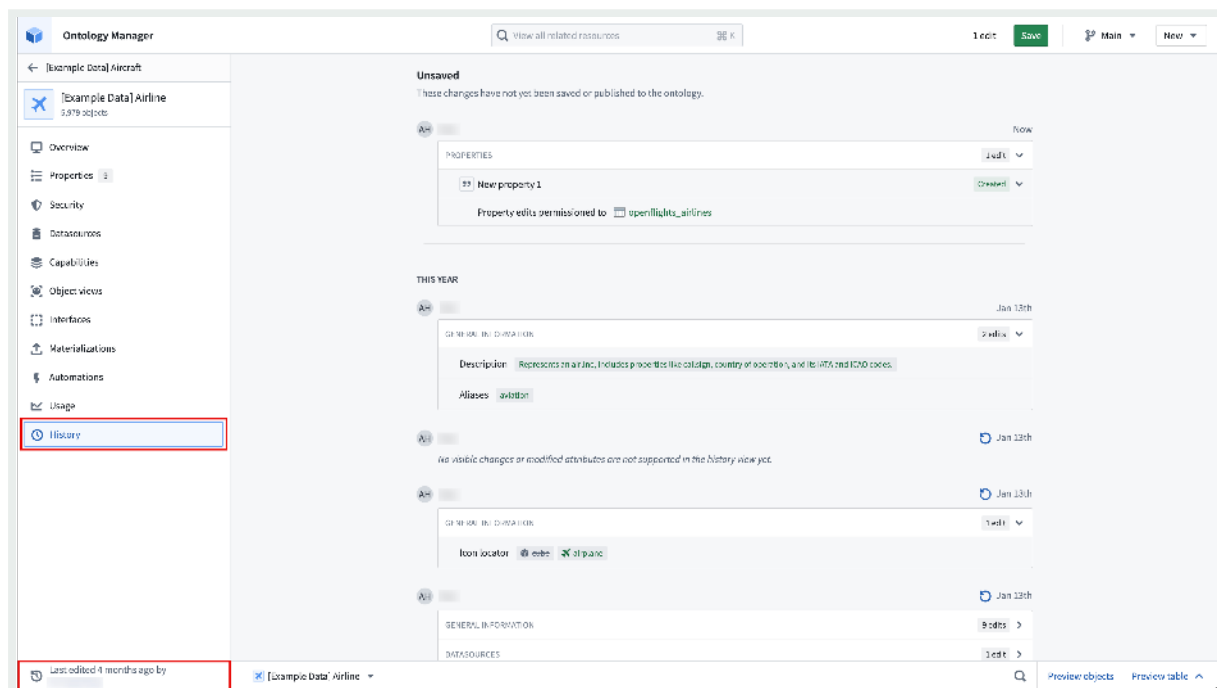
Each entry in the edits history corresponds to a single instance of a user saving changes. You also have the option to consolidate the view by merging changes that have been made by [API Reference](#) into a single entry.

Send feedback

Review changes made to a single Ontology resource

To review changes made to an Ontology resource, select the **History** tab in the Ontology resource's page. A page will be shown with the list of edits made to the resource, including the following details:

- The unsaved changes you made to the resource.
- All saved changes that were made to the resource with details on when the changes were made and the user who applied them.



At the bottom left of an Ontology resource view, a footer states when the resource was last edited and by which user.

Restore changes made to a single object type

Follow the steps below to restore an object type to an older version:

API Reference ↗ button ↶ (anti-clockwise arrow)

2. Select **Confirm**.

Send feedback

Warning

After restoring an object type to a previous version, any changes that were made after the entry you selected will be undone. The changes will be added to your working state and you will need to save your changes to the Ontology for your restore to take effect. [Learn more about saving to the Ontology.](#)

PREVIOUS

← [Save changes to the Ontology](#)

NEXT

[Export, edit, and import an Ontology](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

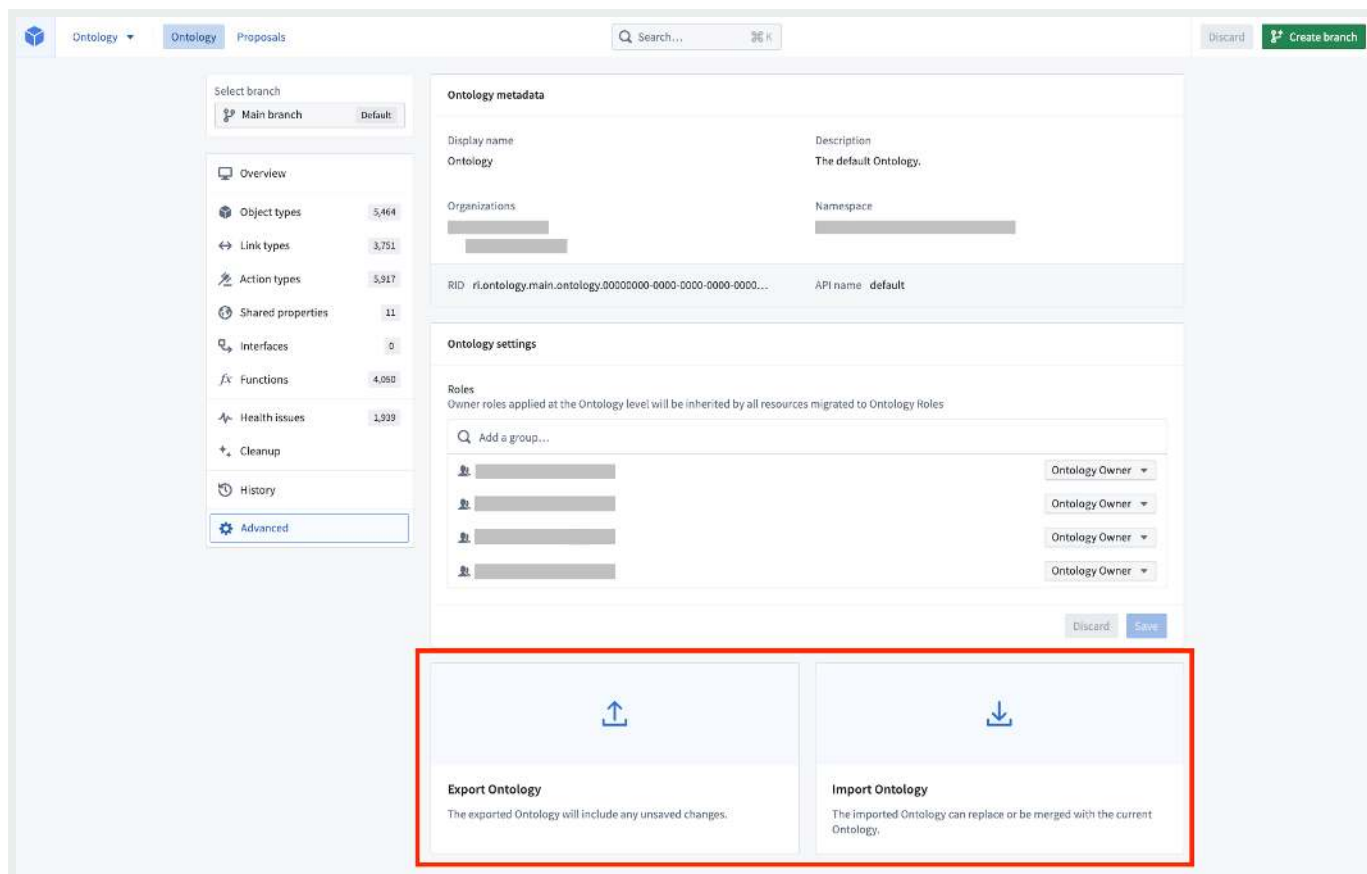
Export, edit, and import an Ontology

i You should not depend on the exported JSON schema as it may change over time.

Ontology schema definitions are stored in a [JSON file ↗](#). An Ontology JSON file can be exported and edited with a code editor or text editor before being imported back into Foundry. This import/export functionality enables two workflows for advanced users:

- If you prefer to make Ontology edits in code, you can bypass the Ontology Manager interface by exporting the Ontology JSON file, editing the JSON file directly in a code editor or text editor, and then importing the modified Ontology JSON file back into the platform.
- If you'd like to copy the working state of one Ontology to another Ontology, you can export the Ontology's current state as a JSON file and then import the copied JSON back into the platform (making any desired changes to the JSON in a code editor).

[API Reference ↗](#)[Send feedback](#)



Export

You can export your Ontology working state by selecting the **Advanced** settings page from the application's home page and then selecting **Export**.

i Any changes you have in your working state will be included in the export.

Import

You can import a previously exported Ontology working state by selecting the **Advanced** settings page from the application's home page and then selecting **Import**. You will be prompted to choose an Ontology file from your local drive.

Next, click **API Reference** which will recreate the entire working state from the JSON file in the application. You will see the number of changes made in the file that need to be saved in the application header.

Send feedback

i An exported Ontology working state with conditional formatting rules configured on its properties cannot be imported to an Ontology other than the one it was exported from.

Troubleshooting

Error: `OntologyMetadata:UnreferencedRuleSets`

If you receive the error `OntologyMetadata:UnreferencedRuleSets`, you are trying to import an Ontology working state with conditional formatting rules that are not defined in that Ontology and cannot be transferred over. You will need to delete the conditional formatting rules from the Ontology working state before importing.

PREVIOUS

← [Change management / Review and restore changes](#)

NEXT

[Ontology cleanup](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Ontology building / Ontology Manager / Ontology cleanup

Ontology cleanup

The Ontology cleanup tool is a safe way to delete object types, and provides a number of benefits, including:

- An easier-to-navigate Ontology without excess object types
- A more performant Ontology as searches and loads are run over fewer object types
- An Ontology without the burden of excess storage costs

The tool aims to help Ontology editors determine the safety of deleting an object type and provides a deprecation option which informs object type users of its future removal.

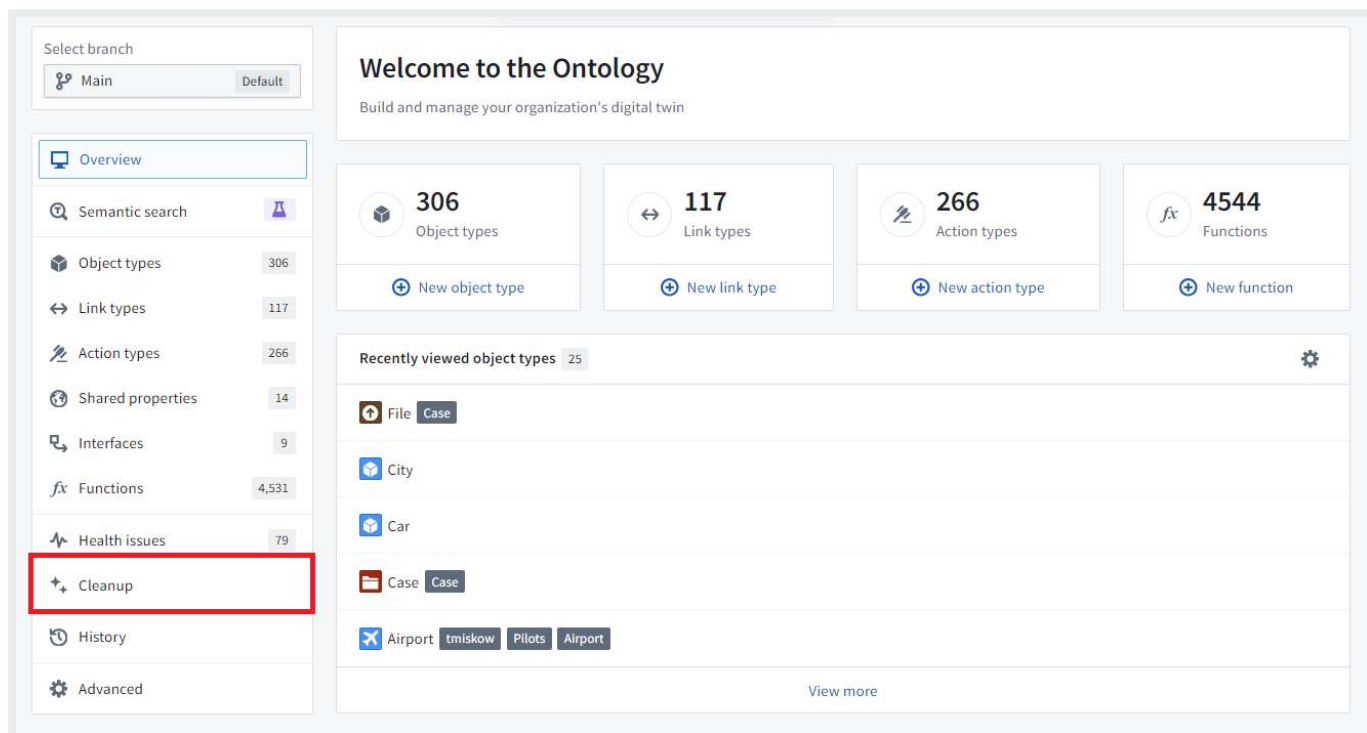
Access Ontology cleanup tool

The view can be accessed from the home page of the Ontology cleanup tool.

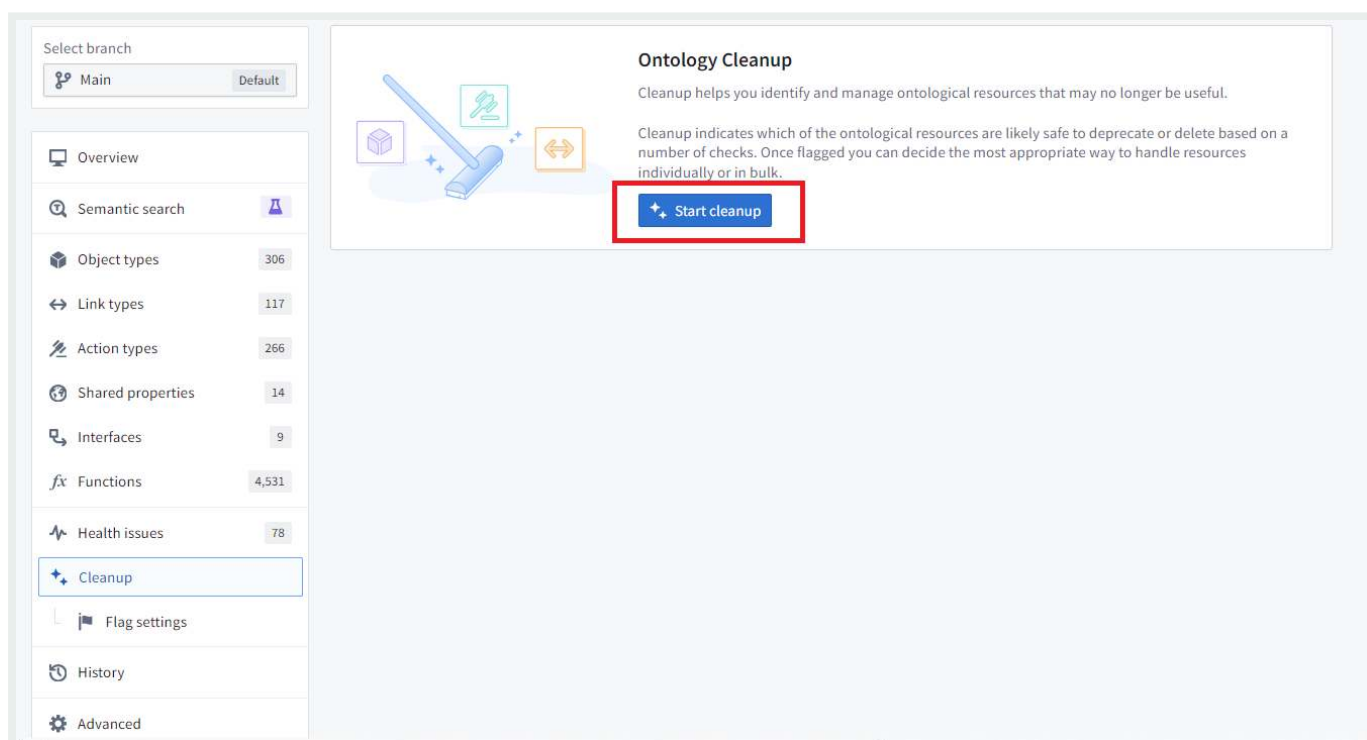
API Reference ↗



Send feedback



When you opt to **Start cleanup**, the tool may take time to find cleanup candidates based on the size of your Ontology.



The resulting list of object types operates similarly to the other pages accessed from the h API Reference ↗ table. The list can be filtered to specific flags or an object type group th ble for. You can also customize which columns are displayed in the table.

Send feedback

Ontology

Archetypes

Ontology

Proposals

Search...

Main

Discard

Create branch

Overview

Semantic search

Object types6,306

Link types4,219

Action types6,600

Shared properties23

Interfaces6

Functions4,684

Health issues1,939

Cleanup5,832/5,833

Flag settings

History

Advanced

Object types5,832/5,833

Snooze

Deprecate

	NAME	GROUPS	PRI...	FLAGS	READS
☐	Cache		Medium	Datasource not updated in 90d	2528
☐	Weather		Medium	Datasource not updated in 90d	2229
☐	Depo		Medium	Datasource not updated in 90d	2184
☐	Customer		Medium	Datasource not updated in 90d	2022
☐	Job		Medium	Datasource not updated in 90d	1401
☐	Revision		Medium	Datasource not updated in 90d	1175
☐	User		Medium	Datasource not updated in 90d	1151
☐	Weather		Medium	Datasource not updated in 90d	10366
☐	Entrance		Medium	Datasource not updated in 90d	8984
☐	Checklist		Medium	Datasource not updated in 90d	7723
☐	Order		Medium	Datasource not updated in 90d	6626

Filters

Flags

☐ No registered usage in 30d3167

☐ Past deprecation date16

☐ Trashed datasource112

☐ Phonograph deindexed735

☐ Datasource not updated in 90d5035

Priority

☐ High

☐ Medium

☐ Low

Actions

☐ Snoozed1

Apply filters

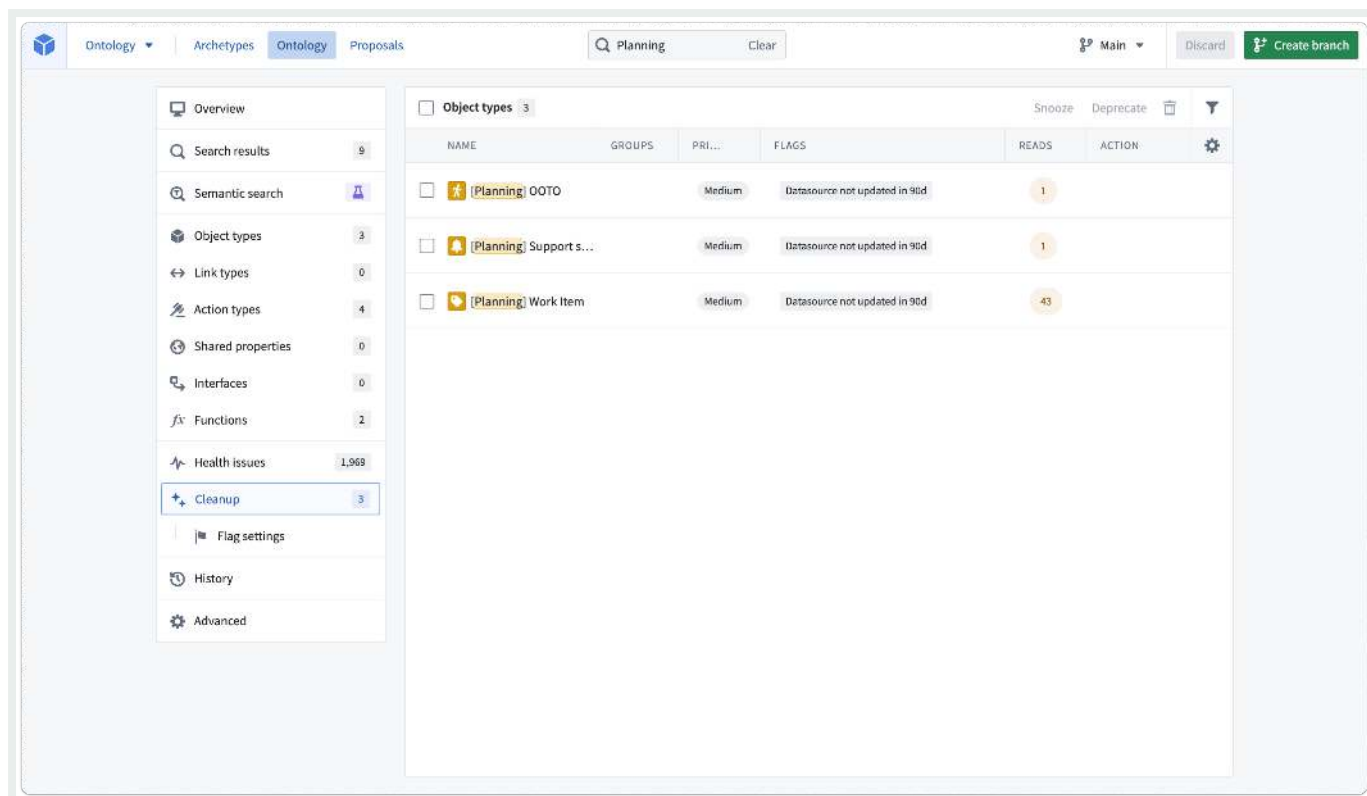
By default, the table is sorted by the highest priority among the flags that an object type triggers.

Cleaning up your Ontology

Here is an example where we have filtered down to the “Planning” workflow, a workflow that was being developed but never released.

API Reference ↗

Send feedback



Select the three object types using the in-line checkboxes.

There are three options for managing these object types:

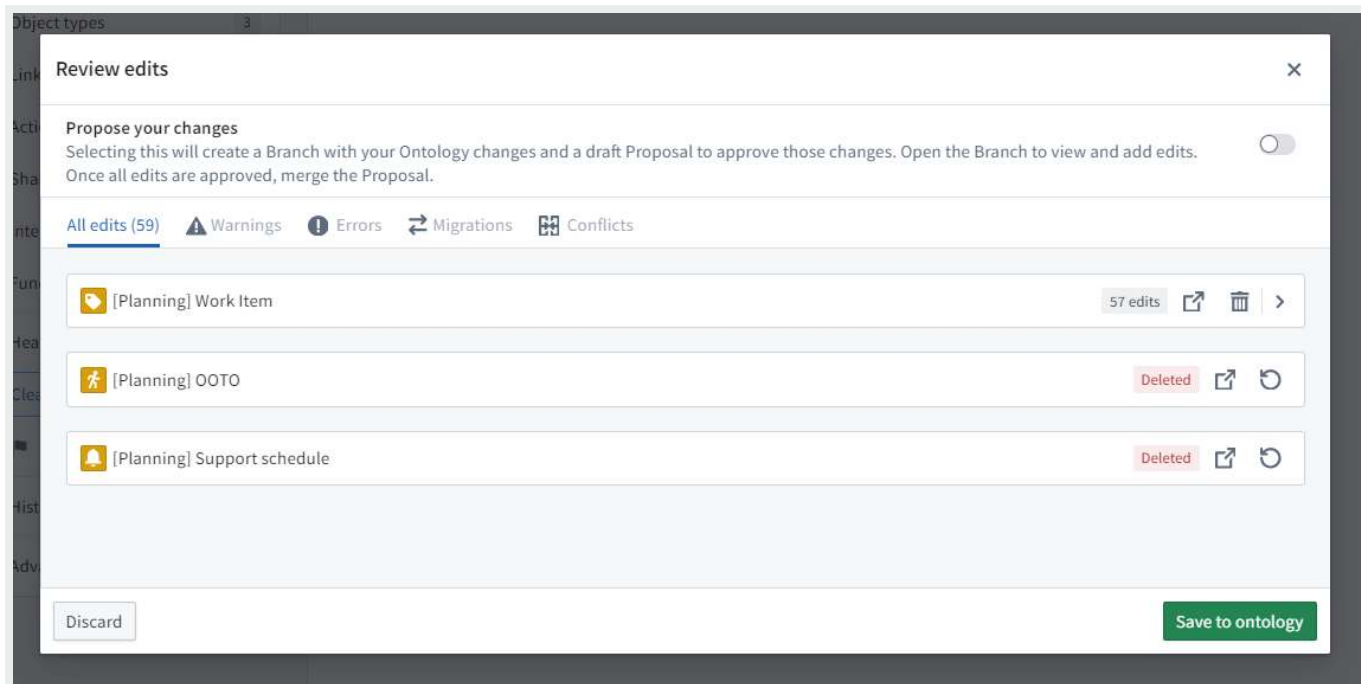
- **Snooze:** Hide object types from your cleanup queue for a configurable amount of time. Snoozing is an action that will affect only the user that performs it.
- **Deprecate:** Show object types as deprecated in every context that displays object type status. This option informs users to move to different object types or flag that the object type is still useful. You can set a deadline along with a deprecation so users know how long they have to refrain from using these object types.
- **Delete:** Delete object types from the Ontology and remove associated data from object storage.

Once you act on an object type in your queue, it disappears from the queue. Use the table filters to view all the actions you have already selected.

Deprecation and deletion are staged the same way as normal Ontology modifications. In the example above, the “Work Item” object type has objects with user edits, so it can be deprecated while the other two deleted. Selecting **Save** in the top right enables saving the changes to the Ontology or creating a proposal to request review from another user.

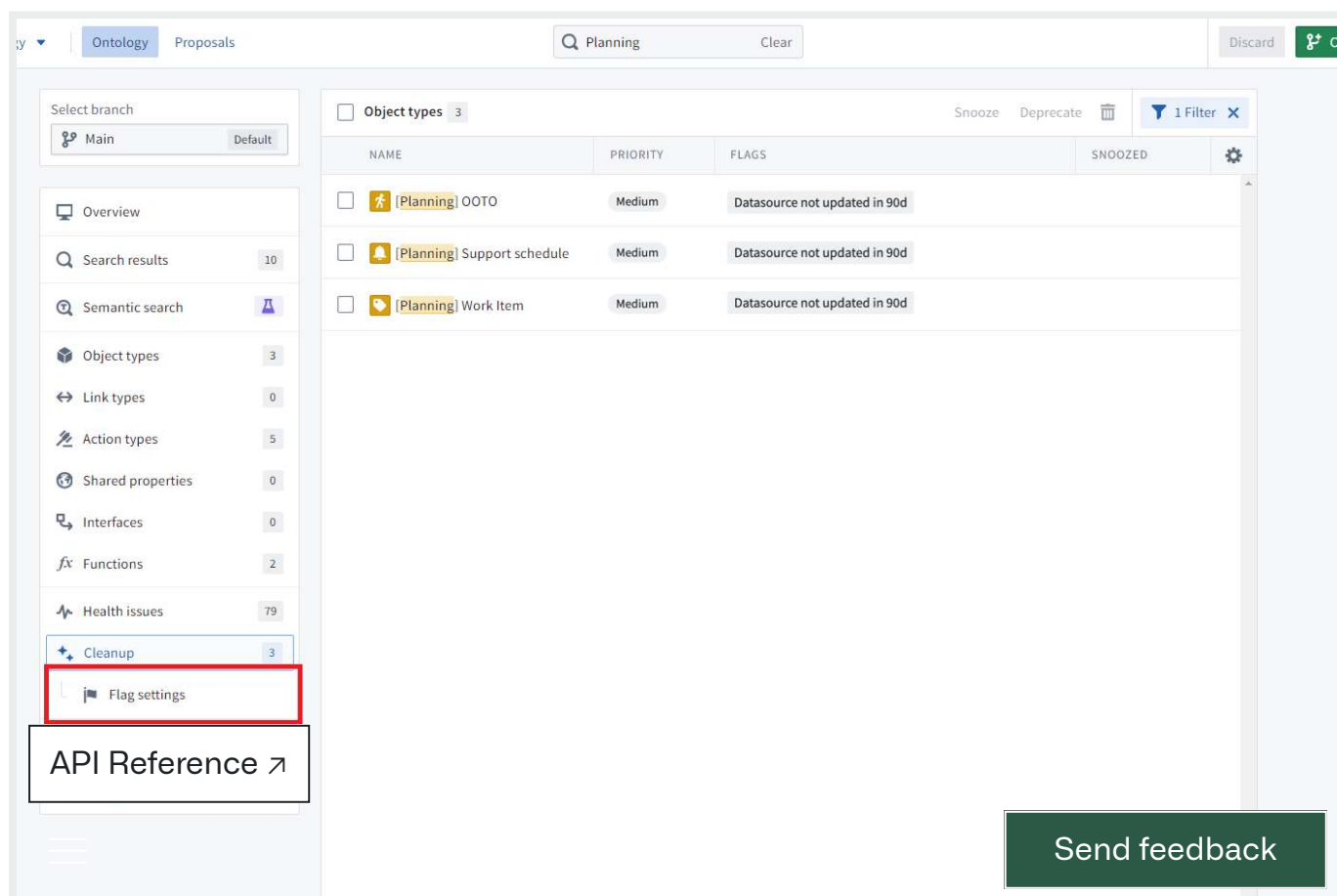
API Reference ↗

Send feedback

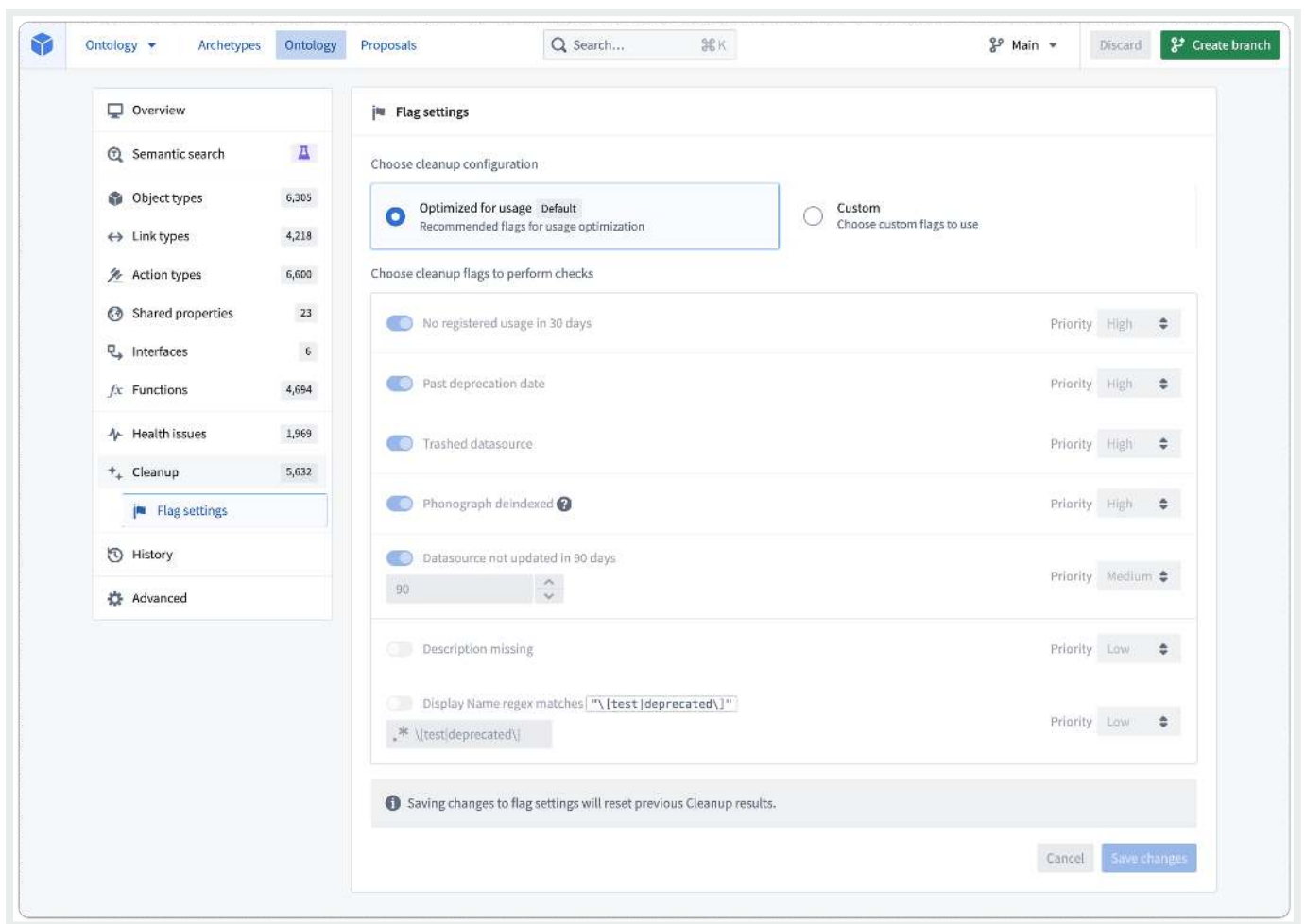


Configure Ontology cleanup

The cleanup page contains a subpage that allows you to customize the flags used and their respective priority.



You can configure flag settings on this page, with a choice of using either the default set or custom flags.



Like snoozing object types from the queue, this is an individual customization that does not affect other Ontology editors.

When you save changes and return to the main **Cleanup** tab, you will be prompted to recalculate the cleanup queue.

Note that if using a custom flag setup, new flags that get added in the future will not be automatically turned on if they are turned on when using the default set of flags.

Ontology cleanup flags

The [API Reference](#) flags is aimed at answering common issues, but is not exhaustive:

- **Past deprecation date:** Object type currently has the deprecated deprecation date field is in the past.

[Send feedback](#)

- **Trashed datasource:** Any datasource (whether dataset, restricted view, or other) backing this object type has been trashed in Compass.
- **Datasource not updated in [x] days:** Checks with Compass the time of the last modification to the backing datasource.
- **Description missing:** The object type has a blank description. Does not check for descriptions on all properties of the object type.
- **Display name regex matches string:** The default value of `\[test|deprecated\]` would match object types that have `[test]` or `[deprecated]` in their display names. For example, if a common pattern at your Organization is to mark object types in user acceptance testing with the prefix `UAT -` or `Testing -`, you would use the regex `UAT -|Testing -` to find all object types matching this pattern. Supports ECMA (JavaScript) regex syntax.
- **Phonograph deindexed:** Flag only applied to object types in Object Storage v1. There is no equivalent check for Object Storage v2.

PREVIOUS

← [Export, edit, and import an Ontology](#)

NEXT

[Vertex / Overview](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)